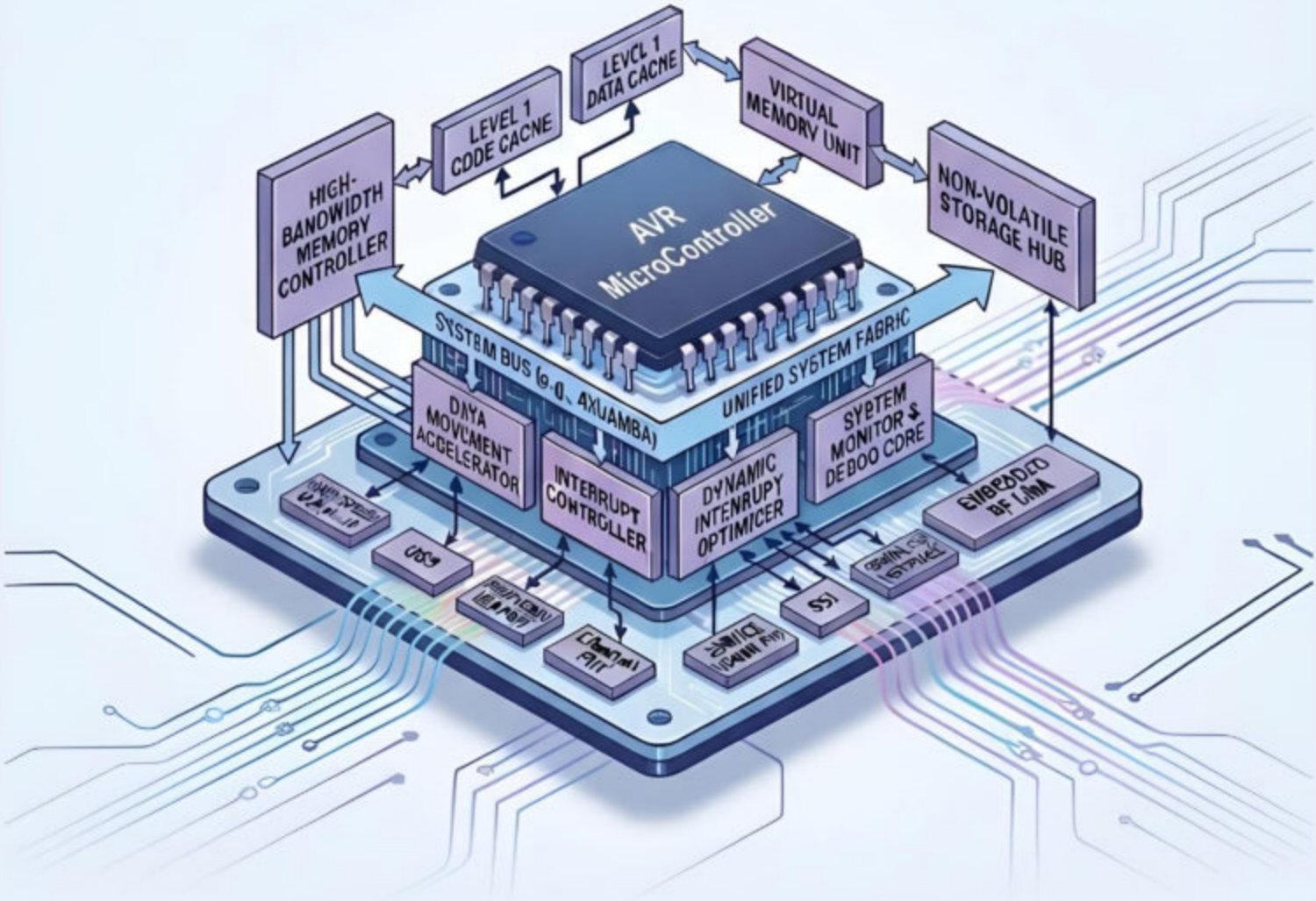
فريق نون التطوعي - السنة الرابعة

المعالجات الصغرية و نظمها

الملحق الأول :

متحكمات الـ AVR

قسم د.ماهر علاوي



كتابة وتدقيق: بلال السابق

تفريغ: سنا ذوالغنى - ليلاس إدلبي

قسم هندسة الالكترونيات والاتصالات

2025 - 2026

المتحكم الصغري MCU :

تعريفه :

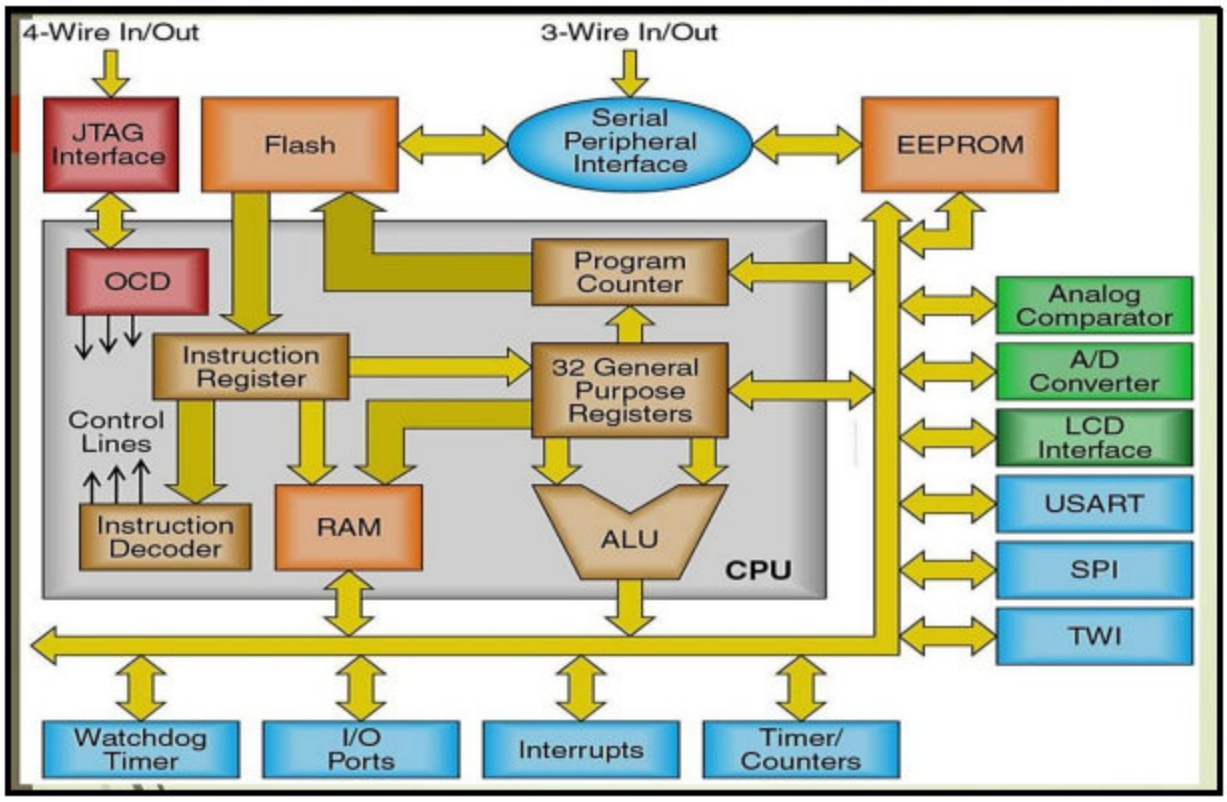
هو شريحة الكترونية متكاملة تحوي كافة العناصر الأساسية للنظام الحاسوبي وهي : CPU ، ذواكر ، I/O Unit ، طرفيات أخرى...

أنواع ذواكر القراءة في المتحكمات :

1. EPROM : يمكن محو محتواها عبر تعريضها للأشعة فوق البنفسجية
2. PROM : قابلة للبرمجة مرة واحدة
3. EEPROM : يمكن محو محتواها كهربائياً

الوحدات الأساسية التي يتألف منها المتحكم:

- ✓ وحدة المعالجة المركزية CPU .
- ✓ ذاكرة الوصول العشوائي RAM .
- ✓ ذاكرة القراءة فقط و تخزين التعليمات ROM .
- ✓ مسجلات خاصة .
- ✓ نظام النواقل Bus_System .
- ✓ محولات D/A و A/D .
- ✓ منافذ الدخل و الخرج .
- ✓ مؤقتات .
- ✓ طرفيات مساعدة أخرى.





والآن يا أصدقاء ...

سنقوم بشرح المكونات في الشكل السابق كلٌّ على حِدة ❤

1. وحدة الحساب و المنطق *ALU* :

من المكونات الأساسية لوحدة المعالجة المركزية ، وظيفتها إجراء العمليات الحسابية و المنطقية .

2. عداد البرنامج *PC (Program Counter)* :

عداد موجود ضمن وحدة المعالجة المركزية ، تشير محتوياته إلى عنوان الأمر الذي حان دور تنفيذه ، عدد بتاته يتعلق بحجم ذاكرة البرمجة فمثلاً إذا كانت الذاكرة 32KB كما في متحكم Atmega328 يكون عدد بتاته 16 بت.

3. مسجل الأوامر *(Instruction Register)* :

من مكونات CPU ، يحتوي شيفرة الأمر الذي يتم تنفيذه حالياً و عدد بتاته يساوي عدد بتات مسار البيانات (8 بت بحالة متحكم 8 بت) وهذا المسجل يتصل بذاكرة البرنامج اللحظية كما هو مبين في الشكل السابق .

4. محلل شيفرة الأوامر *(Instruction Decoder)* :

يتصل بخرج مسجل الأوامر و يفك شيفرة هذه الأوامر لتنشيط خطوط التحكم المناسبة لكل أمر .

5. مسجل الحالة *SR (Statue Register)* :

أحد مسجلات CPU داخل المتحكم ، و يحتوي مجموعة الأعلام (Flags) التي تبين حالة آخر عملية حسابية أو منطقية تم تنفيذها عن طريق ALU .

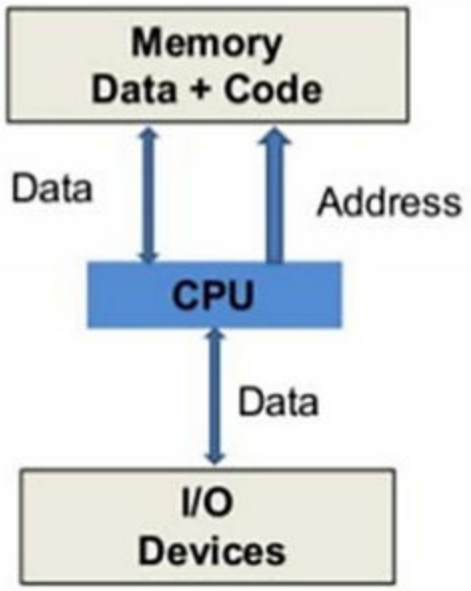
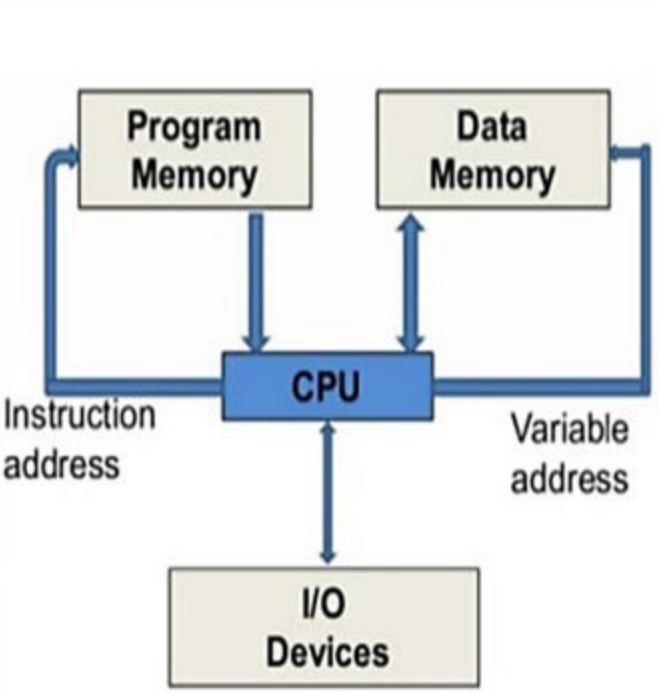
6. المسجلات العامة *(General Purpose Registers)* :

عددها 32 مسجل في إصدارات متحكمات AVR جميعها ، و عدد بتاتها يساوي عدد بتات مسار البيانات (8 بت في متحكمات 8 بت) ، يمكن التعامل مع هذه المسجلات بأسمائها بدءاً من R0 و حتى R31 .

7. الذاكرة *(Memory)* :

إما أن تكون ذاكرة برمجة أو ذاكرة بيانات ، كل من نوعي الذاكرة موصول إلى وحدة المعالجة المركزية بطريقة هارفرد بحيث يتم الحفاظ على تنفيذ معظم أوامر المتحكم في دورة ساعة واحدة .

مقارنة بين بنيتي Von Neumann و Harvard :

_ Von Neumenn Machine _	_ Harvard Machine _
	
✓ ذاكرة واحدة مشتركة تحتوي على البرنامج والبيانات معاً ، مع ناقل (Bus) واحد فقط لنقل البيانات و التعليمات .	✓ ذاكرتان منفصلتان ، واحدة للبرنامج وأخرى للبيانات ، مع ناقلين منفصلين (Buses) يسمحان بالوصول للذاكرتين بنفس الوقت .
✓ تصميم أبسط و أرخص لوجود ناقل واحد فقط.	✓ تصميم أعقد و أكثر تكلفة بسبب وجود ناقلين .
✓ أي خطأ في البرنامج قد يكتب فوق التعليمات ويسبب تعطل البرنامج .	✓ لا يمكن للبرنامج تعديل نفسه .
✓ الذاكرة متاحة للبرنامج و التعليمات معاً.	✓ حجم الذاكرتين قد يختلف

أولاً : ذاكرة البيانات Data Memory :

تفقد محتوياتها بانقطاع القدرة على المتحكم ، و تقسم إلى خمسة أقسام في جميع إصدارات متحكمات AVR كما يلي:

مجموعة المسجلات العامة :

عددها 32 مسجل في متحكمات AVR وعدد بتاتها يساوي عدد بتات مسار البيانات و يمكن التعامل مع هذه المسجلات بأسمائها بدءاً من R0 وحتى R31 ، تبدأ بالعنوان 00H وتنتهي بالعنوان 1FH ؛ (H=HEX) .

مسجلات الإدخال و الإخراج :

عددها تقريباً 64 مسجل وهي خاصة بالتعامل مع الأجهزة الملحقة بالمتحكم مثل المؤقتات و بوابات الإدخال و الإخراج و المحول التماثلي و الرقمي و غير ذلك ..
هذه المسجلات يمكن التعامل معها كعناوين ذاكرة بدءاً من العنوان 20H حتى عنوان آخر مسجل في هذه المسجلات ، أو يمكن التعامل معها بأسمائها الخاصة مثل المسجل DDRD .

ملاحظة : لماذا 20H ؟ لأن المسجلات العامة انتهت عند 1FH ما يكافئ العدد العشري 31 بالتالي ستبدأ هذه المسجلات من العنوان 32 وهو ما يكافئ بالسداسي عشر 20H.

الذاكرة الستاتيكية الداخلية *SRAM* :

هي ذاكرة متطايرة تتراوح ما بين 128 بايت و 4KB حسب إصدار المتحكم ، وتبدأ عناوينها بعد آخر عنوان في مسجلات الإدخال و الإخراج السابقة ، تستخدم بشكل أساسي كمكدس stack عند استدعاء أي برنامج فرعي ، أو عندما يقفز المتحكم إلى برنامج لخدمة مقاطعة معينة ليحتفظ بعنوان الأمر الذي كان ينفذه حتى يعود إليه مرة ثانية بعد الانتهاء من تنفيذ برنامج المقاطعة ، وكذلك تستخدم هذه الذاكرة لتخزين أي بيانات مرحلية أثناء تنفيذ البرنامج.

الذاكرة الستاتيكية الخارجية :

يتم توصيلها عن طريق المستخدم من خارج المتحكم -إن كان المتحكم يدعم ذلك- ، و تبدأ عناوين هذا الجزء من الذاكرة بعد آخر عنوان في الذاكرة الستاتيكية الداخلية .

الذاكرة غير المتطايرة *EEPROM* :

تتراوح بين 64B و 4KB وتستخدم في تخزين الثوابت التي يُخشى من فقدانها في حال انقطاع القدرة ، عناوين هذه الذاكرة غير متتابعة بعد الذاكرة الستاتيكية السابقة لكنها تبدأ من العنوان 000H الخاص بها .

ثانياً : ذاكرة البرمجة (Program Memory):

عبارة عن كمية من الذاكرة اللحظية Flash Memory غير المتطايرة ، تستخدم في كتابة البرامج و تتراوح بين 1KB و 128KB حسب إصدار المتحكم .

- 8. بوابات إدخال و إخراج البيانات .
- 9. وحدة المقاطعة *Interrput Unit* .
- 10. مؤقت *Watchdog Timer* :

هو مؤقت له وظيفة خاصة ، حيث نقوم بتفعيل هذا المؤقت في بداية البرنامج و ضبطه على زمن معين مساوٍ تقريباً لزمن تنفيذ البرنامج وفي نهاية البرنامج نضع أمر يعمل على تصفير المؤقت و منعه من الفيضان فإذا حدث أي شيء غير طبيعي أثناء تنفيذ البرنامج فإن عملية التنفيذ لن تصل إلى أمر تصفير المؤقت و ستستمر عملية العد التصاعدي و بالتالي فيضان للمؤقت ، عندها يقوم المؤقت بإعادة تشغيل المتحكم و بذلك يحمي البرنامج من أي أخطاء قد تتفاقم أثناء عملية التنفيذ .

- 11. وحدات ملحقة أخرى :

مثل المؤقتات و ADC ووحدة الاتصالات المتتالية .

الأنظمة المدمجة Embedded Systems :

تعريفها :

أنظمة رقمية كاملة يجتمع فيها الكيان الصلب Hardware مع العتاد البرمجي Software في بيئة واحدة لتحقيق التكامل المطلوب، حيث يقود العتادُ البرمجي فيها العتادَ الصلب من أجل تنفيذ المهمات المطلوبة منه ، وذلك ضمن نطاق متخصص من التطبيقات..

بعض تطبيقات الأنظمة المدمجة :

أجهزة التكييف ، التلفزيونات ، السيارات ، إشارات المرور، الهواتف النقالة و غيرها..

ونلاحظ أن عدد الأنظمة المضمنة يفوق عدد أجهزة الحواسب بمرات عديدة و التي تشكل 5% فقط من سوق استهلاك المعالجات .

خواص النظم المدمجة :

- النظام المدمج لا بد أن يحتوي على حاسب ، وهذا الحاسب يمكن أن يكون في صورة معالج أو متحكم أو معالج للإشارات الرقمية DSP .
- النظام المدمج يتحكم في تشغيل نظام أو جهاز واحد فقط بعينه و لا يستخدم لأي أغراض أخرى .
- النظام المدمج ذاتي التشغيل بمجرد توصيله بالقدرة الكهربائية ولا يحتاج تدخلاً بشرياً لتحميل برنامج مثلاً.
- مستخدم النظام المدمج لا يستطيع الإخبار بوجود معالج أو متحكم بداخل هذا النظام من عدمه .

عند تصميم الأنظمة المدمجة تؤخذ الاعتبارات التالية :

- المعالجة ضمن الزمن الحقيقي : أي أن الأنظمة تتفاعل مع الأحداث لحظة وقوعها .
- إن أخطاء البرمجة في الأنظمة المدمجة لها عواقب أشد خطورة من تلك التي في الحواسب الشخصية .
- ضوابط معينة تتعلق باستهلاك الطاقة ، مثلاً يمكن أن يكون النظام بعيداً عن مصادر التغذية العامة لذا يجب توفير استهلاكه للطاقة .
- يجب أن تكون الأنظمة المدمجة قادرة على العمل في أحوال جوية ومناخية و بيئية قاسية من حرارة و رطوبة و غبار و...الخ.
- يكون للأنظمة المدمجة عدد أقل من الأجهزة المحيطية ؛ لأن النظام المدمج مصمم لإنجاز هدف محدد بعينه .
- تستخدم أدوات وطرق متخصصة لزيادة وثوقية النظام .

مراحل تصنيع النظام المدمج :

1. تحديد الوظائف المطلوبة من النظام مع وصف دقيق لكل وظيفة.
2. إنشاء نموذج الكيان الصلب ونموذج الكيان البرمجي .
3. اختبار التصميم عن طريق إحدى بيئات المحاكاة .
4. يتم بناء النظام بشكله الأخير ، و يُختبر بشكل عملي قبل طرحه في الأسواق .



سؤال 1 :

ما الفرق بين المعالجات و المتحكمات الصغرية ؟

المعالجات الصغرية *MPU* :

هي ما تمثل وحدة المعالجة المركزية ضمن أي نظام حاسوبي ، فهو عبارة عن دارة متكاملة IC تقوم باستقبال المعطيات و معالجتها و إخراجها على شكل معلومات مفيدة ، لكن يجب تزويد المعالج بطرفيات مثل ببوابات الإدخال و الإخراج ، محول رقمي إلى تماثلي ، محول تماثلي إلى رقمي ، ذواكر و طرفيات أخرى...

المتحكمات الصغرية *MCU* :

عبارة عن دارة متكاملة تحوي بداخلها معالج صغري مع طرفيات أخرى مثل بوابات الإدخال و الإخراج ، محول ADC ، محول DAC ، مؤقتات و ذواكر و...الخ ، وكل ذلك على شريحة واحدة .



سؤال 2 :

هل نستخدم معالج أم متحكم صغري ؟

الجواب :

_ إذا كان التطبيق يتطلب قوة حسابية متقدمة مثل معالجة صور أو قواعد بيانات ، عندها نستخدم معالج منفصل يستطيع التعامل مع الكميات الكبيرة من البيانات .

_ أما إذا كان التطبيق يتطلب سرعة وحجم صغير و استهلاك منخفض للطاقة ولا يحتاج قوة حسابية كبيرة عندها نستخدم متحكمات صغرية .

- مثل التحكم بإيقاف و تشغيل أجهزة معينة .

العوامل المؤثرة في اختيار المعالجات و المتحكمات :

- عدد خطوط إدخال و إخراج البيانات .
- هل البيانات التي سنتعامل معها متوازية أم متتالية .
- سعة الذاكرة المطلوبة .
- عدد خطوط المقاطعة .
- طريقة كتابة البرنامج ، فمن الممكن استخدام لغة التجميع Assemply أو أحد اللغات عالية المستوى مثل C++ أو Basic ..
- المعالجة في الزمن الحقيقي.



مثال لتوضيح المراد من المعالجة في الزمن الحقيقي :

إذا كنا نتعامل مثلاً مع إشارة صوت بتردد 20KHz يجب أن يكون معدل قراءة البيانات 40KHz أو أكثر فيكون الزمن بين كل عينة من هذا الصوت و التي تليها 25µs ، عندها يجب أن يكون زمن معالجة هذه العينة مع زمن تخزينها وإخراجها أقل من 25µs وبمعامل أمان كافٍ وإلا ستفقد الكثير من العينات .

تنفيذ الأوامر في متحكمات AVR

مراحل تنفيذ أي تعليمة :

- ✓ **جلب Fetching**
- ✓ **فهم Decoding**
- ✓ **تنفيذ Execution**
- ✓ **تخزين Storing**

نفرض للسهولة أن هناك دورتين لتنفيذ التعليمات و هي:

- **دورة الجلب Fetching**

المسؤولة عن إحضار الأمر من الذاكرة ووضعه في مسجل الأوامر

- **دورة التنفيذ Execution**

التي تبدأ بعد وصول شيفرة الأمر إلى مسجل الأوامر .



في الأجيال الأولى من المعالجات (8bit) ، عندما تكون الوحدة مشغولة بدورة الإحضار أو التنفيذ فإنها لا تفعل أي شيء آخر ، لذلك فإن الزمن الكلي لتنفيذ أي أمر هو مجموع زمني دورتي الإحضار والتنفيذ معاً .

لكن فيما بعد تم استبدال هذه الفكرة بفكرة خطوط الأنابيب Pipelines ، وهو أسلوب لتنفيذ أكثر من تعليمة بشكل متداخل عوضاً عن تنفيذ كل تعليمة بالكامل قبل الانتقال للتعليمة التالية .

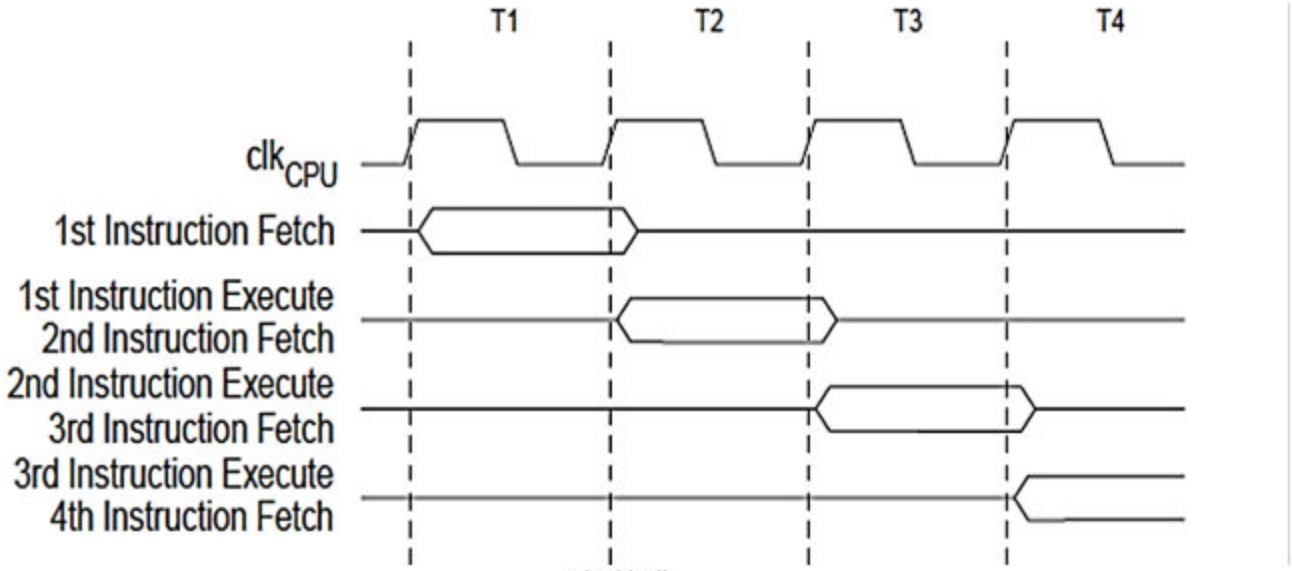
مثال لتوضيح الفرق بين العمل بطريقة pipelines وبدونها :

	بدون pipelines	مع pipelines
دورة 1	جلب I1	جلب I1
دورة 2	تنفيذ I1	تنفيذ I1 + جلب I2
دورة 3	جلب I2	تنفيذ I2 + جلب I3
دورة 4	تنفيذ I2	تنفيذ I3 + جلب I4



نلاحظ أنه بدون Piplines لازال المعالج في الدورة الرابعة يقوم بتنفيذ التعليمة الثانية بينما وصل إلى جلب التعليمة الرابعة مع Pipelines .

التتابع الزمني لدخول الأوامر في خط الأنابيب pipelines



الشكل (1)

تعليمات CISC و RISC

- CISC : Complex Instruction Set Computer .

- RISC : Reduced Instruction Set Computer .

إن زمن تنفيذ أي برنامج يمكن وضعه في المعادلة التالية:

زمن التنفيذ = عدد الأوامر في البرنامج * عدد نبضات تنفيذ كل أمر * زمن كل نبضة

في *CISC* :

- يتم تقليل زمن تنفيذ البرنامج عن طريق تقليل عدد الأوامر في البرنامج، وذلك بجعلها أكثر تعقيداً.
- أوامر مركبة يستغرق تنفيذها عدة نبضات .
- تشتمل الأوامر على عمليات التخزين والاستدعاء من الذاكرة .
- عدد الأوامر في البرنامج يكون أقل ، لكن كل أمر يحتاج للعديد من نبضات الساعة.
- الأوامر تنفذ في أزمنة مختلفة اعتماداً على نوع الأمر .
- من الصعب تنفيذها باستخدام طريقة Pipelines .
- لا تحتاج الكثير من المسجلات العامة .

مثال على تنفيذ عملية ضرب وفق تعليمات CISC : MUL a , b

في *RISC* :

- يتم تقليل زمن تنفيذ البرنامج عن طريق تقليل عدد النبضات لكل أمر .
- أوامر أولية يحتاج كل منها نبضة تزامن واحدة للتنفيذ .
- التخزين والاستدعاء من الذاكرة يكون بأمرين منفصلين .
- عدد الأوامر في البرنامج يكون أكثر ، لكن كل أمر يحتاج لنبضة ساعة واحدة فقط للتنفيذ.
- جميع الأوامر تقريباً تنفذ في أزمنة متساوية .
- يمكن تنفيذها بطريقة Pipelines لأن أزمنة تنفيذ الأوامر متساوية.
- تحتاج الكثير من المسجلات العامة .

مثال على تنفيذ عملية ضرب وفق تعليمات RISC :

Load A , a

Load B , b

MULT A , B

Store a , A



ملاحظة : معظم المتحكمات ومنها متحكمات AVR تعتمد معمارية RISC .

نظرة موسعة على منتجات شركة ATMEL من متحكمات AVR :

مجموعة متحكمات *ATtiny AVR* :

- ✓ تتميز هذه المجموعة بأنها الأقل من حيث حجم ذاكرة البرمجة و عدد أطراف إدخال و إخراج البيانات .
- ✓ تحتوي هذه المجموعة على حوالي 40 إصدار.

مجموعة متحكمات *ATmega AVR* :

- ✓ تتراوح كمية ذاكرة البرمجة من 4KB حتى 128KB.
- ✓ عدد أطراف إدخال و إخراج البيانات يتراوح من 23 حتى 53 طرف.
- ✓ تحتوي هذه المجموعة على حوالي 41 إصدار كلها 8bit .

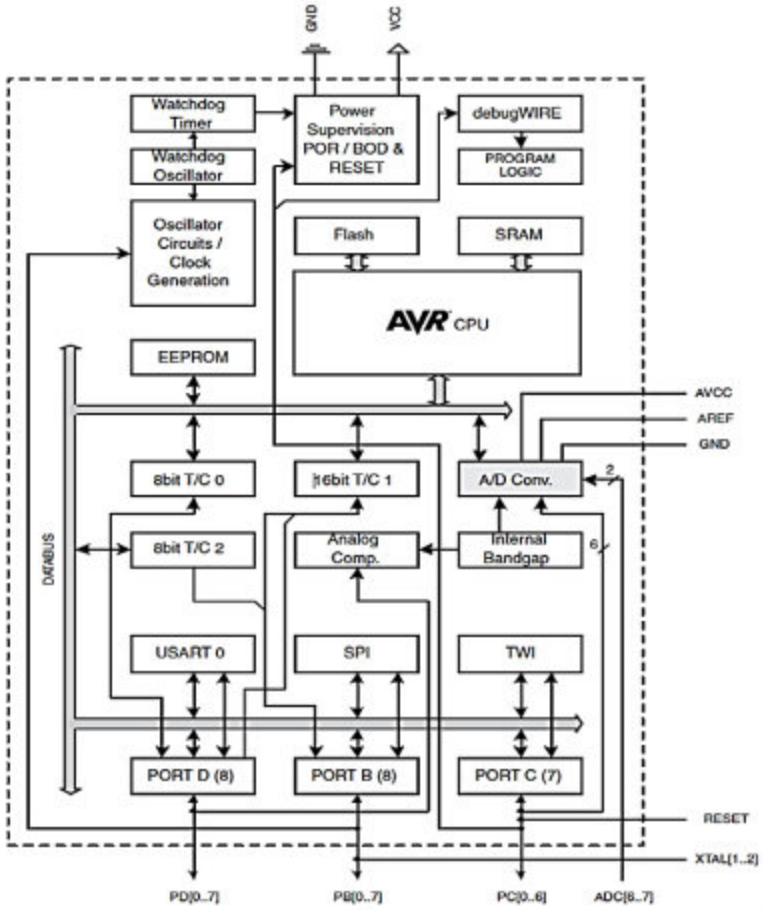
مجموعة متحكمات *AVRXMEGA* :

- ✓ تتراوح ذاكرة البرمجة من 16KB حتى 256KB .
- ✓ تتراوح الذاكرة العشوائية بين 2KBحتى 16KB
- ✓ عدد أطراف هذه الشرائح من 44 حتى 100 طرف ، منها 34 حتى 78 طرفاً لإدخال و إخراج البيانات .
- ✓ إصدارات هذه المجموعة ذات 8Bit أو 16Bit عالية الإمكانات .

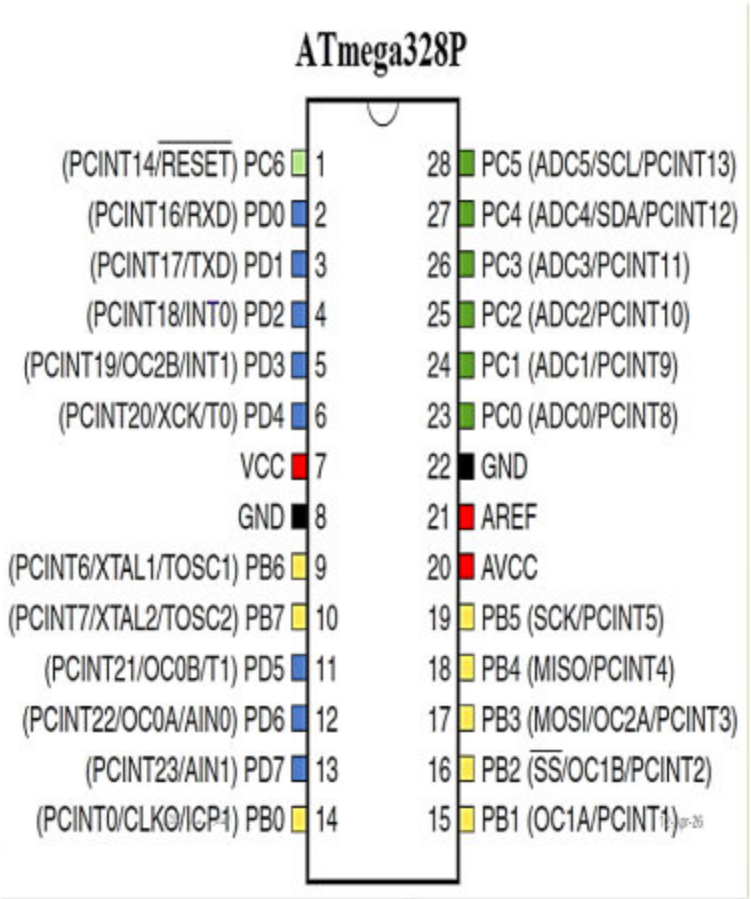
- مجموعة متحكمات *AVRUC3* :

- ✓ ذاكرة البرمجة و الذاكرة العشوائية كبيرة .
- ✓ تتراوح عدد أرجل هذه الشرائح من 48 حتى 144 طرف ، بالتالي عدد كبير من أطراف إدخال و إخراج البيانات .
- ✓ جميع إصداراتها ذات 32bit .
- ✓ إمكانياتها عالية جداً .

المتحكم ATMEGA_328 :



الشكل (3)



الشكل (2)

ملاحظة :

الصور للإطلاع فقط و غير مطلوبة للحفظ و إنما لمعرفة شكل المتحكم و أطراف الإدخال و الإخراج و التغذية ..

المزايا و البنية الداخلية لشريحة المتحكم ATmega 328 :

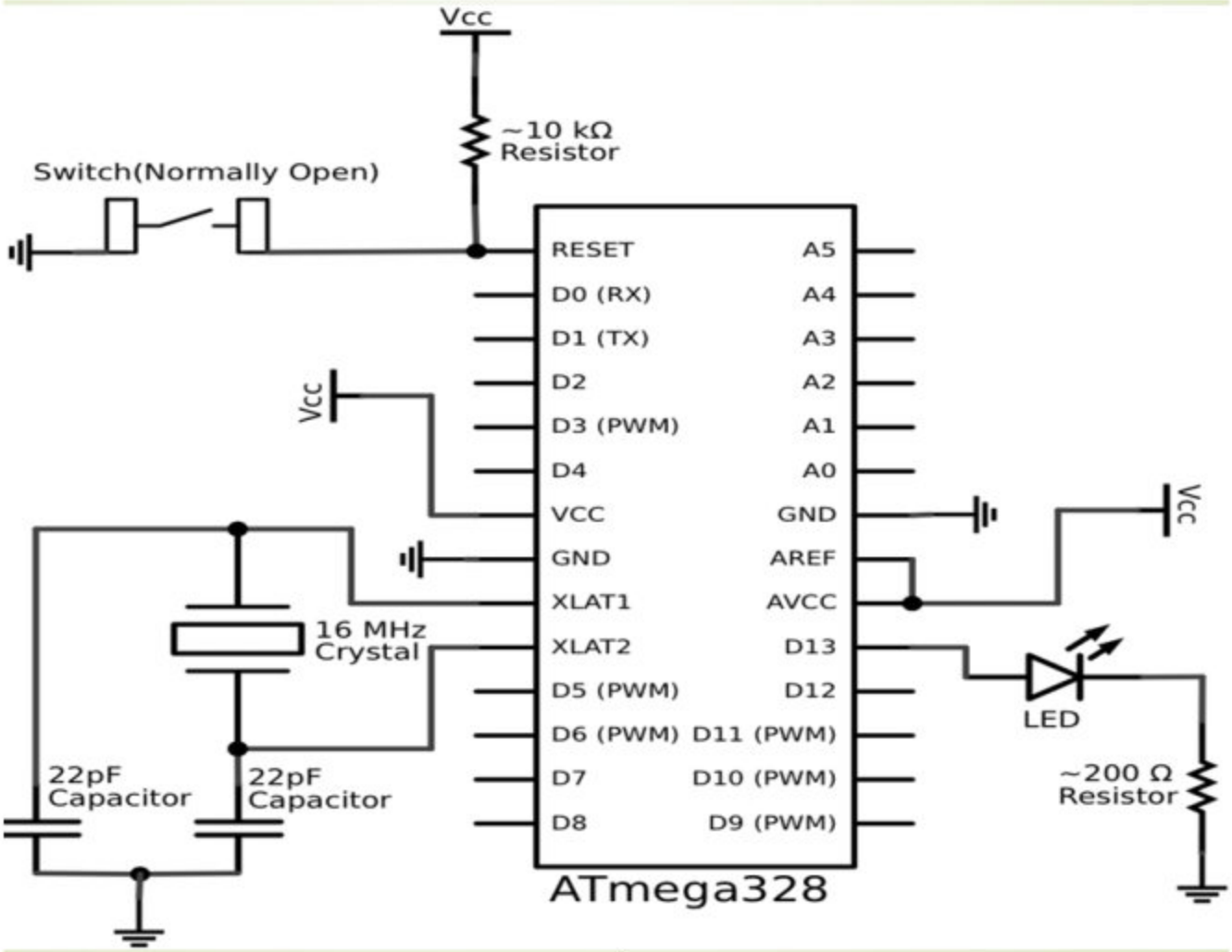
- الشريحة مجهزة بـ32 من المسجلات العامة وكلها 8bit .
- يتبع هذا المتحكم تصميم هارفرد لتوصيل ذاكرة البرمجة و البيانات على وحدة المعالجة المركزية .
- هذا المتحكم يتبع معمارية RISC و عدد أوامر التجميع الأولية هو 131 .
- حجم ذاكرة EEPROM هو 1KB.
- حجم ذاكرة البرمجة Flash هو 32KB ،

ويمكن برمجة المتحكم وهو في التطبيق ISP: In System Programming

- حجم الذاكرة العشوائية SRAM هو 2KB .
- يوجد في الشريحة مؤقتين 8bit ومؤقت واحد 16bit .
- الشريحة بها ساعة للزمن الحقيقي بمذبذب خاص بها للحصول على الوقت و التاريخ
- تحتوي ستة قنوات لتعديل عرض النبضة PWM
- وجود مؤقت Watchdog.
- إمكانية التعامل مع عدة أنواع لتبادل البيانات التتابعية من خلال USART و I2C و SPI
- خطين للمقاطعة الخارجية ، و إمكانية المقاطعة عند حدوث أي تغيير .
- إمكانية التعامل مع نبضات تزامن داخلية أو خارجية
- 23 خط لإدخال و إخراج البيانات ، يمكنها أن تسحب أو تعطي تيار حتى 20mA.
- جهد تشغيل من 1.8v حتى 5.5v .
- نبضات ساعة تصل إلى 20MHZ .



دارة تشغيل بسيطة للمتحكم ATmega_328 :



الشكل (4)

إدخال و إخراج البيانات للمتحكم ATmega_328 :

المقصود من إدخال البيانات : جعل المتحكم يقرأ بيانات من أجهزة موصولة عليه مثل لوحة المفاتيح ، أو حساس حرارة ، و الكثير غيرها ...

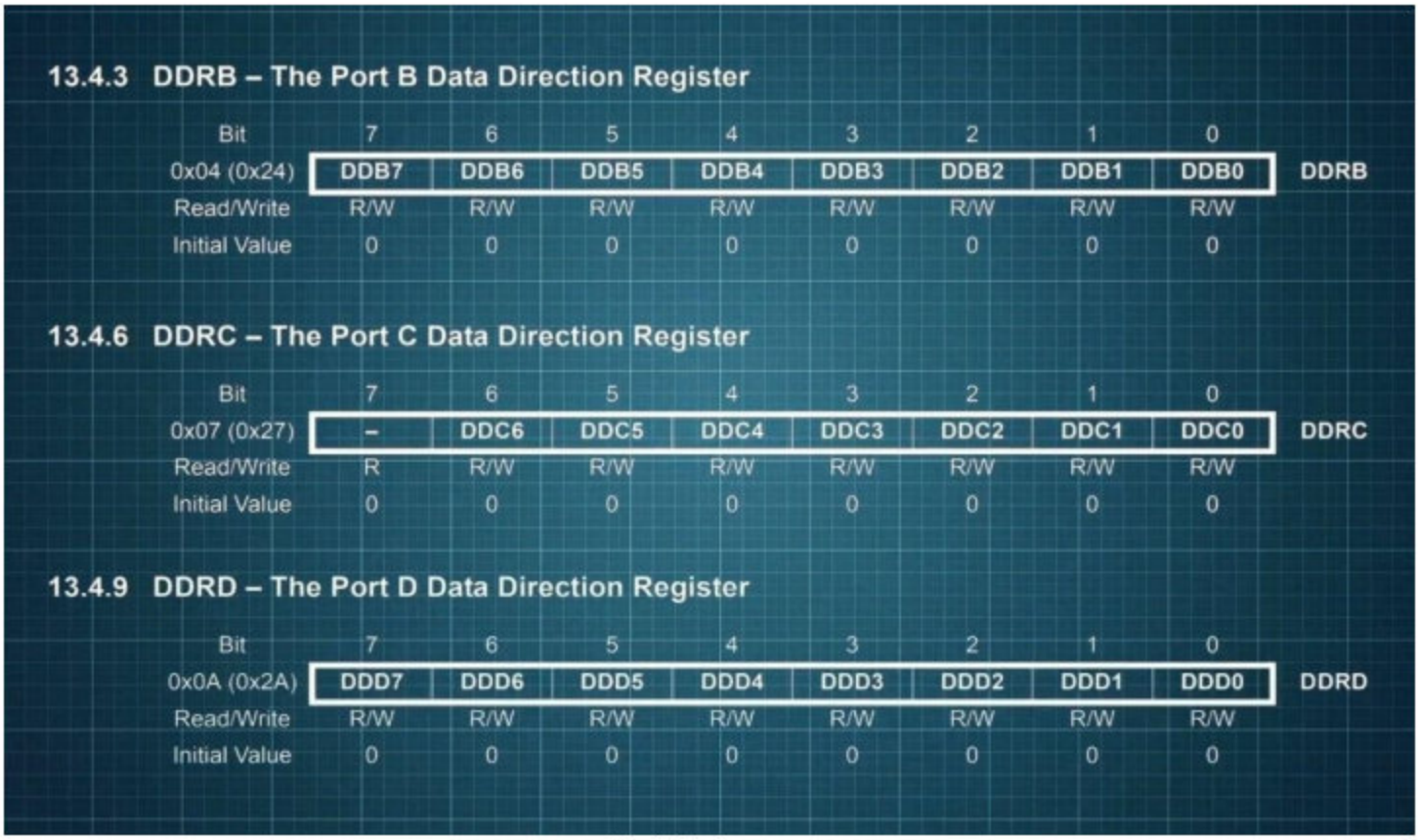
المقصود بإخراج البيانات : إخراج بيانات من المتحكم لأحد الأجهزة الموصولة عليه من أجل التحكم في هذا الجهاز، مثل إخراج إشارة للتحكم بسرعة محرك ، إخراج إشارة إلى شاشة LCD ، و الكثير من إشارات التحكم الأخرى .

نلاحظ من المخطط السابق الذي يوضح البنية الداخلية للمتحكم وجود ثلاث بوابات لإدخال و إخراج البيانات الرقمية وهي البوابات B , C , D ذات الـ23 منفذاً (8+7+8 = 23) يمكن برمجة كل طرف (أو منفذ PIN) من هذه المنافذ ليصبح خط إدخال أو إخراج للبيانات ويتم التعامل مع كل واحدة من هذه البوابات من خلال مسجلات ..

مسجلات DDRX , PORTX , PINX

المسجل DDRX (Data Direction Register) :

هو مسجل اتجاه البيانات لأي بوابة X من البوابات الثلاث فإذا كانت قيمة الخانة المقابلة للمنفذ تساوي 1 يصبح المنفذ بحالة إخراج ، و إذا كانت قيمة قيمة الخانة المقابلة 0 يصبح المنفذ بحالة إدخال .



13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.6 DDRC – The Port C Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x07 (0x27)	–	DDC6	DDC5	DDC4	DDC3	DDC2	DDC1	DDC0	DDRC
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.9 DDRD – The Port D Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x0A (0x2A)	DDD7	DDD6	DDD5	DDD4	DDD3	DDD2	DDD1	DDD0	DDRD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

الشكل (5)

الصورة السابقة - الشكل(5) - من Datasheet الخاصة بالمتحكم ATmega_328

وتمثل المسجلات الثلاث :

- DDRB : المسؤول عن تعيين منافذ البوابة B كمداخل و مخارج .
- DDRC : المسؤول عن تعيين منافذ البوابة C كمداخل و مخارج .
- DDRD : المسؤول عن تعيين منافذ البوابة D كمداخل و مخارج .

حجم المسجلات 8bit مرقمة من 0 إلى 7 من اليمين إلى اليسار كما هو موضح في الصورة كل بت يتحكم بمنفذ معين في البوابة ..

بالعودة إلى الشكل(2) نلاحظ أن المنافذ باللون الأصفر هي الخاصة بالبوابة B .

فمثلاً إن أردنا تعيين المنافذ PB0 و PB1 و PB4 كمخارج و بقية منافذ البوابة كمداخل نقوم بتخزين القيم التالي في المسجل DDRB :



7	6	5	4	3	2	1	0
0	0	0	1	0	0	1	1

وذلك عن طريق الأمر البرمجي :

```
DDRB = OB00010011;
```

حيث أن OB تعني أن الرقم المكتوب بعدها بالصيغة الثنائية و يتألف من 8 خانات كل خانة تقابل بت في المسجل .

ويمكن كتابة نفس الأمر السابق بوضع الرقم بصيغة سداسي عشر Hex :

```
DDRB = OX13;
```

ويدل الرمز OX على العدد السداسي عشر (Hex) المكتوب بعدها

مثال 2 : نريد تعيين المنافذ PC2 و PC3 و PC4 كمخارج :

```
DDRC = OB00011100 ;
```

المسجل *PORTX* :

يحتوي البيانات المطلوب إخراجها على البوابة X

Set pin HIGH or LOW with registers

13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.5 PORTC – The Port C Data Register

Bit	7	6	5	4	3	2	1	0	
0x08 (0x28)	–	PORTC6	PORTC5	PORTC4	PORTC3	PORTC2	PORTC1	PORTC0	PORTC
Read/Write	R	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.8 PORTD – The Port D Data Register

Bit	7	6	5	4	3	2	1	0	
0x0B (0x2B)	PORTD7	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0	PORTD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

الشكل (6)

في المثال السابق قمنا بتعيين المنافذ PC2 و PC3 و PC4 كمخارج عن طريق مسجل DDRC، والآن نريد إخراج القيمة 1 على المنفذ PC2 والقيمة 0 على المنفذين PC3 و PC4 فنخزن القيم التالية في المسجل :



7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0

وذلك عن طريق الأمر البرمجي :

```
PORTC = 0B00000100;
```

أو بصيغة Hex :

```
PORTC = 0X4;
```

مثال آخر : نريد إخراج القيمة F0 على أطراف البوابة D :

```
DDRD = 0XFF ;          // تعيين أطراف البوابة D كأطراف خرج
```

وضع أربع أصفار على الأطراف الأربعة الأولى ووضع أربع واحدات على الأطراف الأربعة الثانية من البوابة D

```
PORTD = 0XF0 ;         //
```

المسجل *PINX* :

يحتوي هذا المسجل على البيانات المدخلة على أطراف أي بوابة بعد أن تم تشكيلها كبوابة دخل .

13.4.4 PINB – The Port B Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

13.4.7 PINC – The Port C Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x06 (0x26)	–	PINC6	PINC5	PINC4	PINC3	PINC2	PINC1	PINC0	PINC
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	0	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

13.4.10 PIND – The Port D Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x09 (0x29)	PIND7	PIND6	PIND5	PIND4	PIND3	PIND2	PIND1	PIND0	PIND
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

الشكل(7)

مثال على ذلك :



```
DDRD = 0X00 ;

DDRB = 0XFF ;

n = PIND ;

PORTB = n ;
```

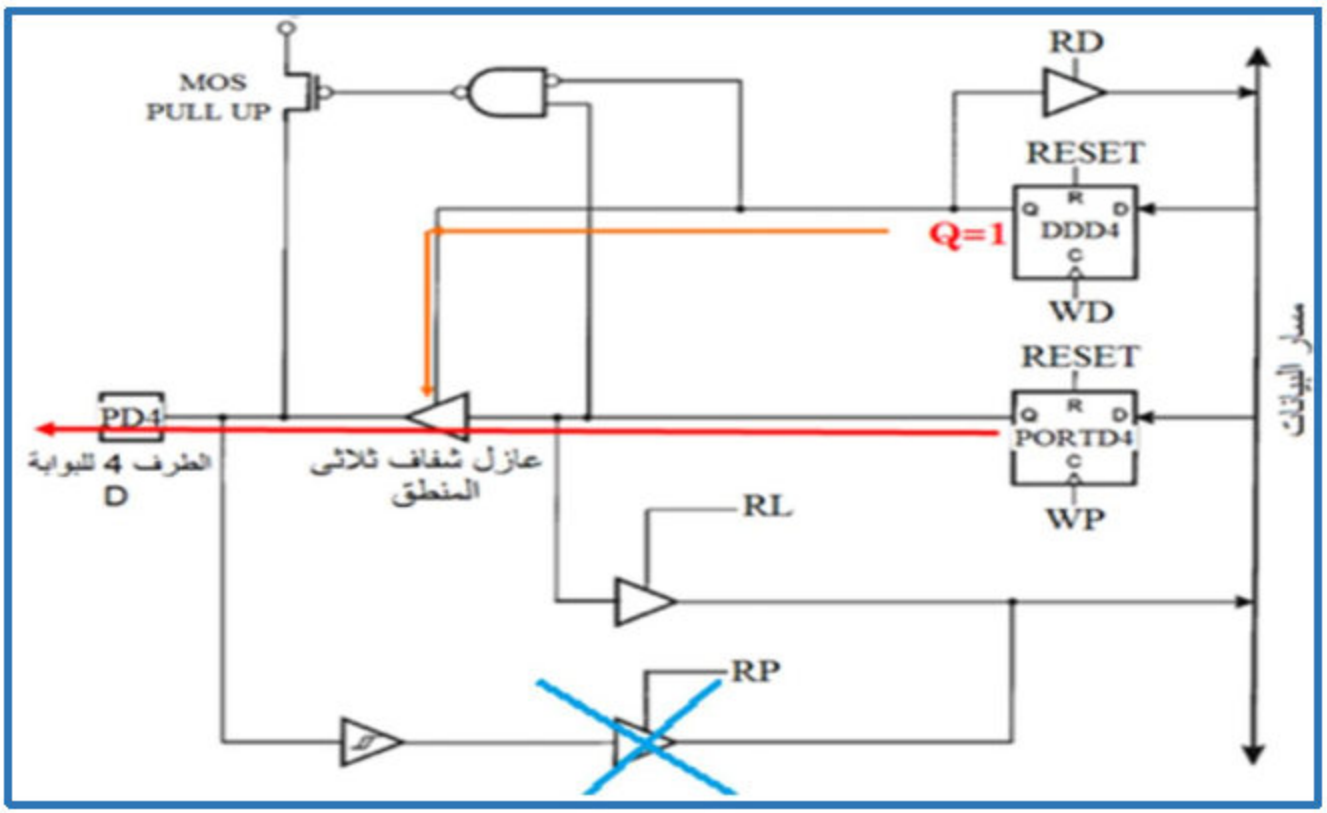
- الأمر الأول يشكل أطراف البوابة D لتكون جميعها أطراف دخل .
- الأمر الثاني يشكل أطراف البوابة B لتكون جميعها أطراف خرج .
- الأمر الثالث يقرأ القيم على البوابة D ويضع محتوياتها في المتغير n .
- الأمر الرابع يخرج محتويات المتغير n على أطراف البوابة B .



تشكيل أي طرف من أطراف المتحكم كطرف خرج (هاردويرياً)

يبين الشكل (8) طريقة تشكيل الطرف 4 من أطراف البوابة *D* كطرف خرج ، كمثال على أي طرف آخر من أطراف أي بوابة في المتحكم .

- ✓ نضع 1 في البت المقابل لهذا الطرف في مسجل DDRD أي DDD4=1 وهذا البت عبارة عن قلاب FlipFlop أي عندما نضع فيه 1 سيصبح خرجه Q=1
- ✓ خرج القلاب متصل بخط تحكم العازل الشفاف ثلاثي المنطق ، بالتالي عندما يكون 1 سيكون العازل موصل (خرجه يساوي دخله) أي أنه سيجعل خرج بت مسجل الخرج PORTD4 موصلاً على طرفي الخارجي PD4 بالتالي فإن أي بيانات يتم وضعها في البت PORTD4 سيتم رؤيتها مباشرة على طرف البوابة الخارجي PD4 .
- ✓ بتنشيط الخط RD يمكن قراءة محتويات القلاب DDD4 وهناك أوامر للتنفيذ ذلك .
- ✓ بتنشيط الخط RL يمكن قراءة خرج القلاب PORTD4 وهناك أوامر لتنفيذ ذلك .
- ✓ الخط RP سيكون مساوياً للصفر عند تشكيل الطرف كخرج بالتالي سيكون العازل الشفاف الموجود في طريق القراءة غير موصل .

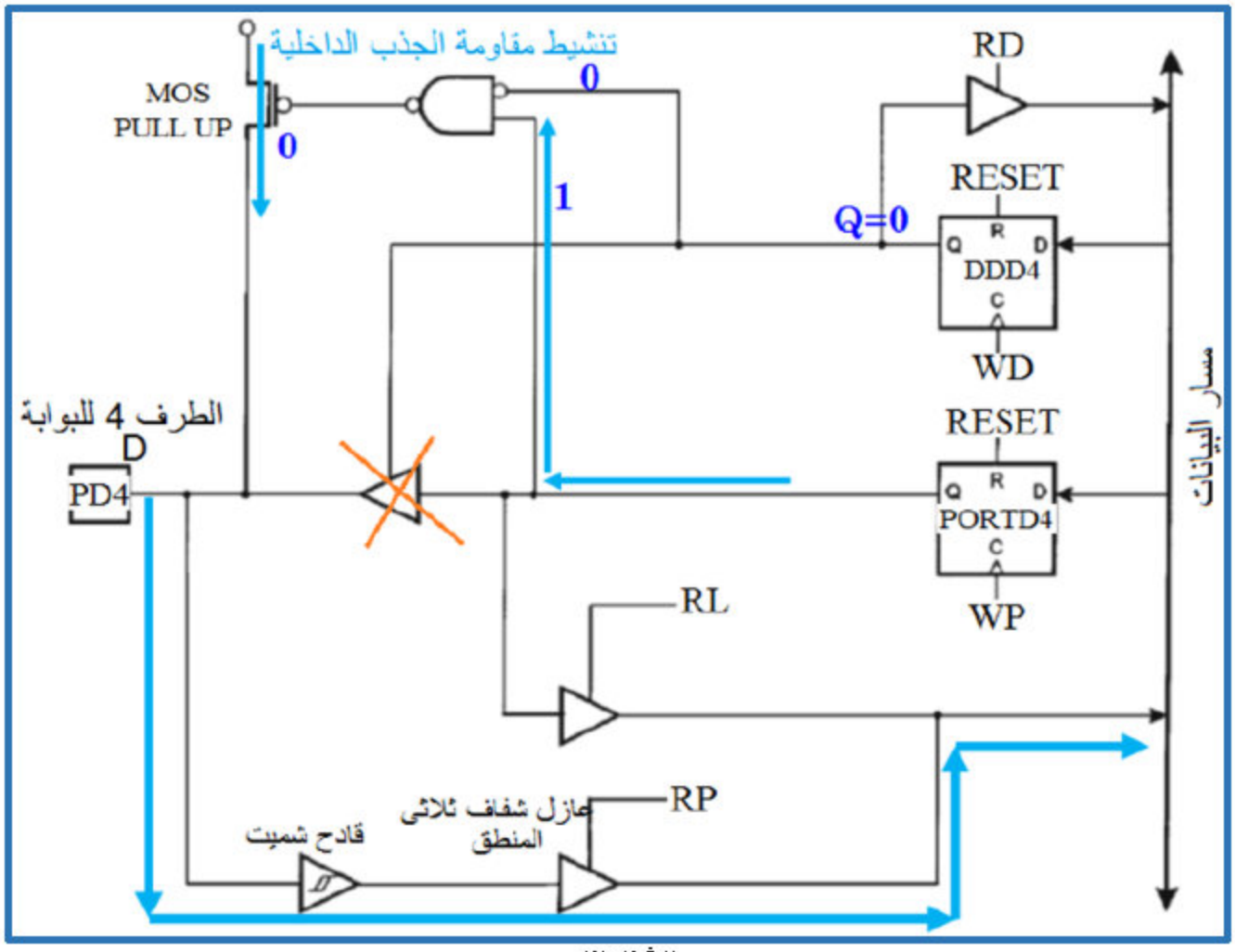


الشكل (8)

تشكيل أي طرف من أطراف المتحكم كطرف دخل (هاردويرياً) :

يبين الشكل (9) طريقة تشكيل الطرف 4 من أطراف البوابة *D* كطرف دخل كمثال .

- ✓ نضع 0 في البت المقابل لهذا الطرف في مسجل DDRD أي DDD4=0 وهذا البت عبارة عن قلاب FlipFlop أي عندما نضع فيه 0 سيصبح خرجه Q=0
- ✓ خرج القلاب متصل بخط تحكم العازل الشفاف ثلاثي المنطق بالتالي عندما يكون 0 سيكون العازل مفتوح ، بالتالي خرج بت مسجل الخرج PORTD4 غير موصول إلى الطرف الخارجي PD4
- ✓ بتنشيط الخط RP يمكن قراءة البيانات من الطرف PD4
- ✓ لتنشيط مقاومة الرفع الداخلية Pull UP نلاحظ وجود MOSFET وبوابة NAND مسؤولان عن ذلك ، ولتفعيل الـMOSFET يجب أن يكون DDD4=0 لأنه موصول إلى أحد مداخل بوابة NAND عبر بوابةNOT ونضع 1 في البت PORTD4 فيعمل الـMOSFET وتنشط مقاومة الرفع الداخلية



الشكل (9)

برمجة ومحاكاة متحكمات AVR :

هناك الكثير من برامج تحرير الأكواد مثل Arduino IDE ، Atmel Studio

و برامج محاكاة مثل Protues

الشكل العام لكتابة برنامج التحكم C :

```c
#include <avr/io.h>

int main(void)
{

    while (1)
    {
        /*application code here */
    }
}
```

التطبيق الأول:

كود يقوم بقراءة البيانات الموجودة على منافذ البوابة B ووضعها على منافذ البوابة D :

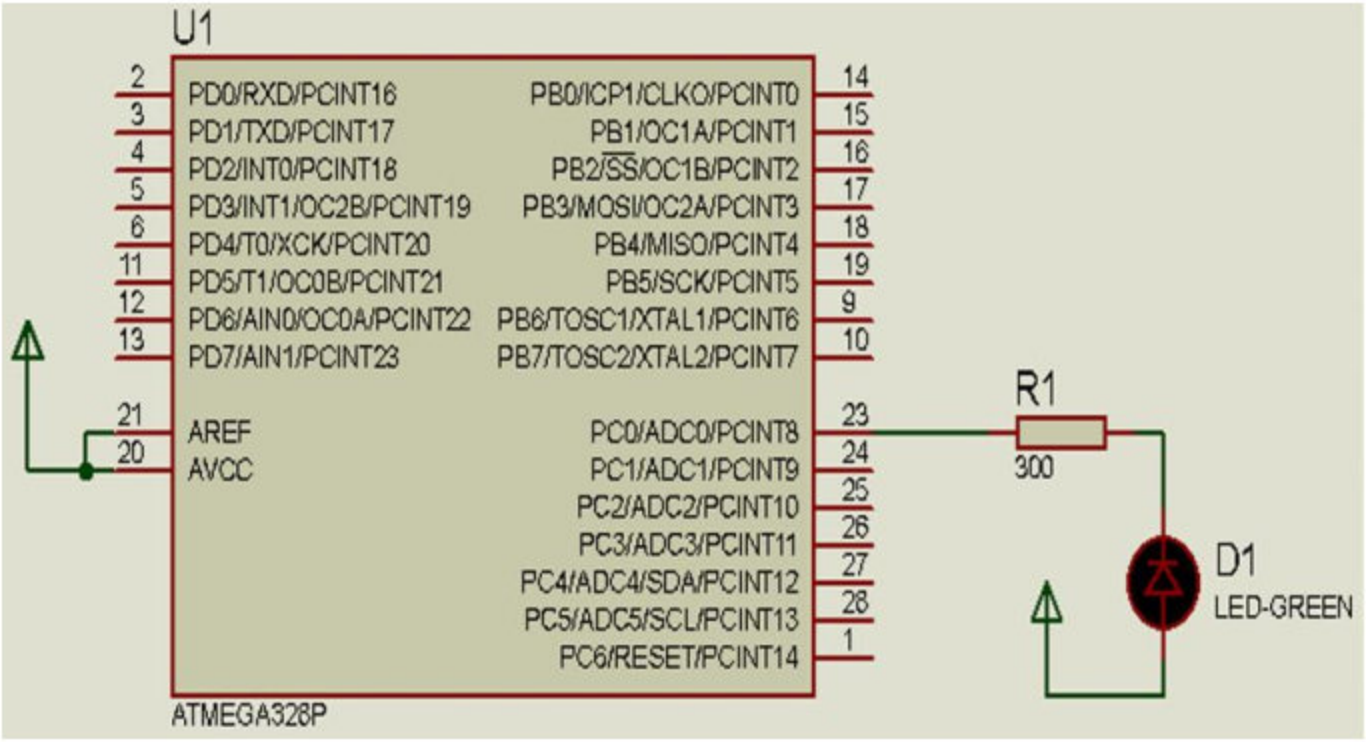
```c
#include <avr/io.h>

int main(void)
{
    unsigned char n;
    DDRB=0x00;  // Make Port B as input
    DDRD=0xFF; // Make Port D as output

    while (1)
    {
        n=PINB;
        PORTD=n;
    }
}
```

التطبيق الثاني:

لإنارة و إطفار LED بمعدل معين (Blinking)



الكود :

```
#define F_CPU 1000000 //CPU freq in hertz (enable internal oscillator)
#include <util/delay.h> //delay in util library
#include <avr/io.h>

int main(void)
{
    DDRC=0xFF ; // port C as output

    while (1)
    {
        PORTC=0xFF ; // output ones on port C
        _delay_ms(500) ; //delay 500 millisecond (half second)
        PORTC=0x00 ; // zeros on port C
        _delay_ms(500) ; //delay half second and repeat
    }
}
```

شرح الكود السابق :



1. **السطر الأول تم تحديد تردد المهتز الداخلي 1MHz**
2. **السطر الثاني استدعاء مكتبة التأخير الزمني**
3. **السطر الثالث استدعاء ملف الإخال والإخراج الموجود في مكتبة AVR**
4. **الأمر الأول في دالة main هو DDRC=0XFF يجعل كل أطراف البوابة C أطراف خرج**
5. **الأمر الأول داخل while(1) هو PORTC=0XFF يضع واحدات على كل اطراف البوابة C، ثم delay-ms(500)- تأخير نصف ثانية ثم وضع أصفار على كل طرف من أطراف البوابة C عن طريق الأمر PORTC=0X00 ثم تأخير زمني لمدة نصف ثانية أيضاً .**

ملاحظة : يوجد أمر delay-µs- الذي يعطي تأخيراً بالميكروثانية.

العمليات المختلفة على مستوى البت في لغة C :

رمز العملية	العملية
&	عملية الآند AND، تعطى ١ إذا كان الاثنين بت وحايد، وتعطى صفرا فيما عدا ذلك
|	عملية الأور OR، تعطى صفرا إذا كان الاثنين بت يساويان أصفرا، وتعطى واحد فيما عدا ذلك
^	عملية الإكس أور XOR (هى الرمز الموجود فوق الرقم ٦ على لوحة المفاتيح)، تعطى واحد إذا كان الاثنين بت مختلفين، وتعطى صفرا إذا كان الاثنين بت متساويين
~	عملية العكس NOT، تعكس محتويات أى بت
<<	الإزاحة لليسار، البتات الخارجة من اليسار تهمل أو تضيع، ويتم إدخال أصفار من ناحية اليمين فى البتات الزائدة
>>	الإزاحة ناحية اليمين، البتات الخارجة من اليمين تهمل أو تضيع، ويتم إدخال أصفارا من ناحية اليسار فى البتات الزائدة.

التعامل مع بتات محددة داخل أي متغير

في الأكواد السابقة عند كتابة PORTC=0XF0 أو ما يُكافئ PORTC=0B11110000 فإننا نضع اصفار في البتات الأربعة الأولى و واحدات في البتات الأربعة الأخرى *أي أننا نقوم بضبط كل بتات المسجل معاً*.

لكن ماذا لو أردت ضبط بت واحد فقط ؟

نقوم بإجراء عملية OR للمسجل مع 1 مُزاح إلى اليسار بمقدار رقم البت المطلوب .

مثال1

: لوضع 1 في البت رقم 2 من المسجل PORTC :

```
PORTC |= (1 << 2) ;
```

=| : عملية OR 2>> : إزاحة لليسار بمقدار2

مثال 2:

لوضع 1 في البت 4 من المسجل DDRB :

```
DDRB |= (1 << 4) ;
```

مثال3 :

لوضع 1 في البت صفر من المسجل DDRD :

```
DDRD |= (1 << 0) ;
```

لوضع صفر في بت معين من المسجل نقوم بإجراء عملية AND للمسجل مع معكوس الـ 1 مُزاح إلى اليسار بمقدار مكان البت المطلوب .

ملاحظة : لا نكتب صفر و إنما 1 و إشارة NOT(~) قبلها

مثال1 :

لوضع 0 في البت رقم 3 من المسجل PORTD :

```
PORTD &= ~(1 << 3) ;
```

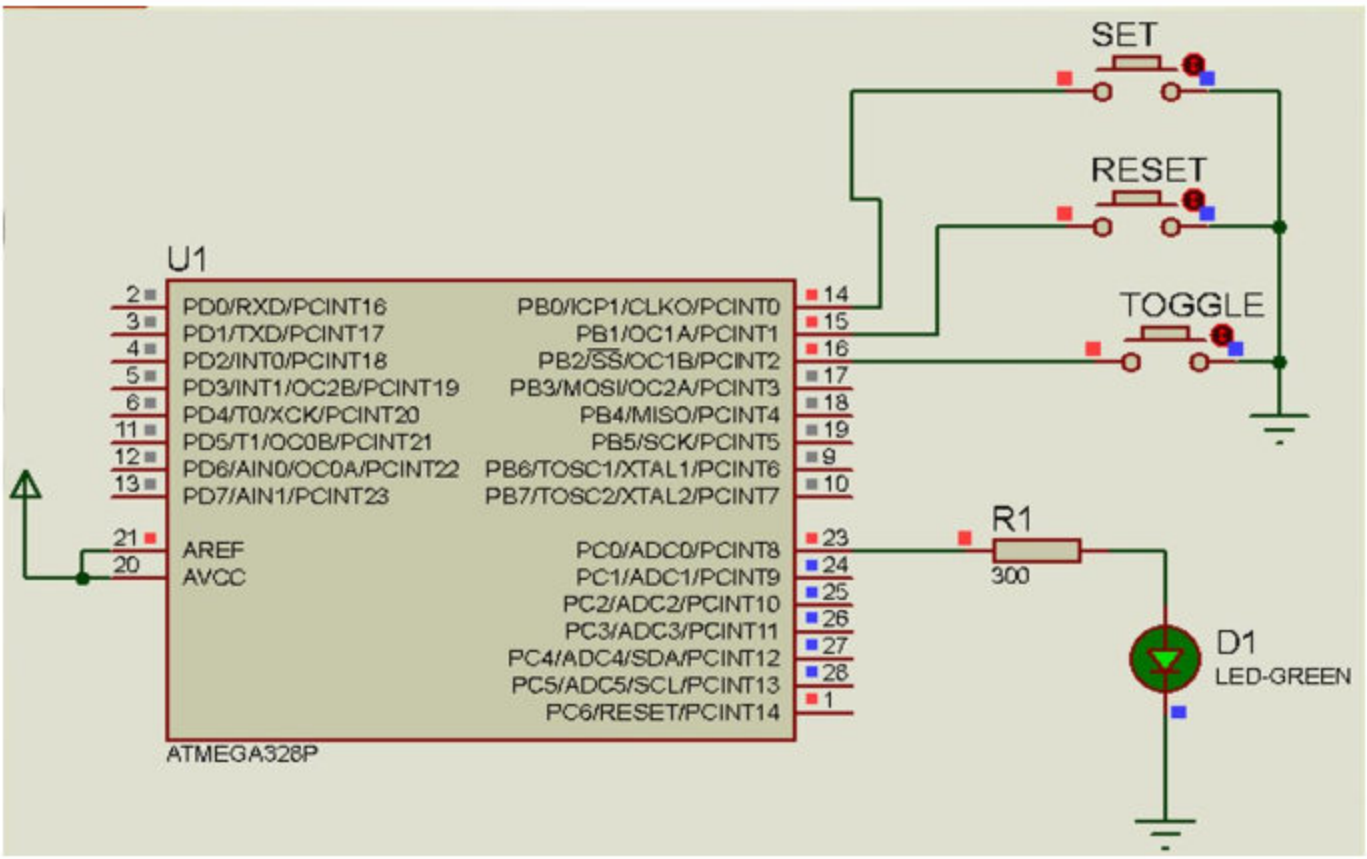
مثال2 :

لوضع 0 في البت رقم 5 من المسجل PORTC :

```
PORTC &= ~(1 << 5) ;
```

التطبيق الثالث :

التحكم بديود ضوئي من خلال ثلاثة مفاتيح :



الكود:

```
#define F_CPU 1000000 //CPU freq in hertz (enable internal oscillator)
#include <avr/io.h>

int main(void)
{
    DDRB=0x00; //port B input
    DDRC=0xFF; //port C output
    PORTB |= (1 << PORTB0)|(1 << PORTB1)|(1 << PORTB2); //Turn on pull-up resistance on the first three inputs.

    while (1)
    {
        if ((PINB & (1<<0))==0)
        {PORTC |= (1<<0);}        //Setting PC0
        if ((PINB & (1<<1))==0)
        {PORTC &=~ (1<<0);}       //Resetting PC0
        if ((PINB & (1<<2))==0)
        {PORTC ^= (1<<0);}        //Toggle PC0
    }
}
```



شرح الكود السابق :

1. تضمين مكتبة avr وتعريف تردد المهتز الداخلي قبل main

2. داخل الmain :

- تحديد منافذ البوابة B كمداخل .

- تحديد منافذ البوابة C كمخارج .

- تفعيل PULL UP على المنافذ PC0 , PC1 , PC2

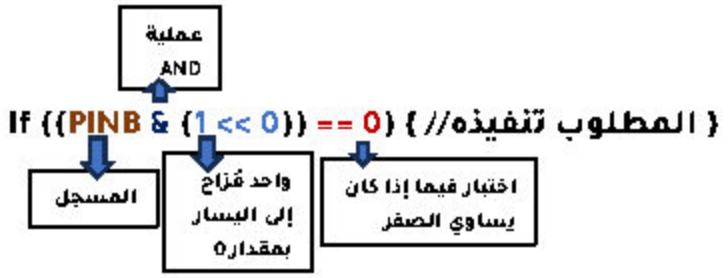
3. داخل While :

نلاحظ من الدارة أن الزر موصول طرفه الآخر إلى الأرضي ، أي عند الضغط عليه سيقرأ مدخل المتحكم صفر وعند عدم الضغط سيقرأ واحد بسبب وجود PULL UP

كيف نتحقق من وجود صفر على منفذ ما ؟

نقوم بعملية AND بين المسجل و 1 مُزاح إلى اليسار بمقدار رقم البت المطلوب التحقق منه ، إذا كان مساوياً للصفر فإن البت الذي نتحقق منه يساوي الصفر فعلاً (عند عمل AND بين أي متغير منطقي والواحد فإن الناتج يكون مساوياً الصفر فقط إذا كان المغير صفراً).

ويتم ذلك من خلال تعليمة الشرط :



بنفس الطريقة نقرأ قيمة المنافذ الثلاث الموصولة على الأزرار لعمل set أو reset أو toggle LED حسب الزر المضغوط .

ملاحظة : عند تفعيل PULL UP على المداخل قمنا بكتابة (PORTB0 << 1) بدل (0<< 1) و (PORTB1<<1)بدل (1<<1)

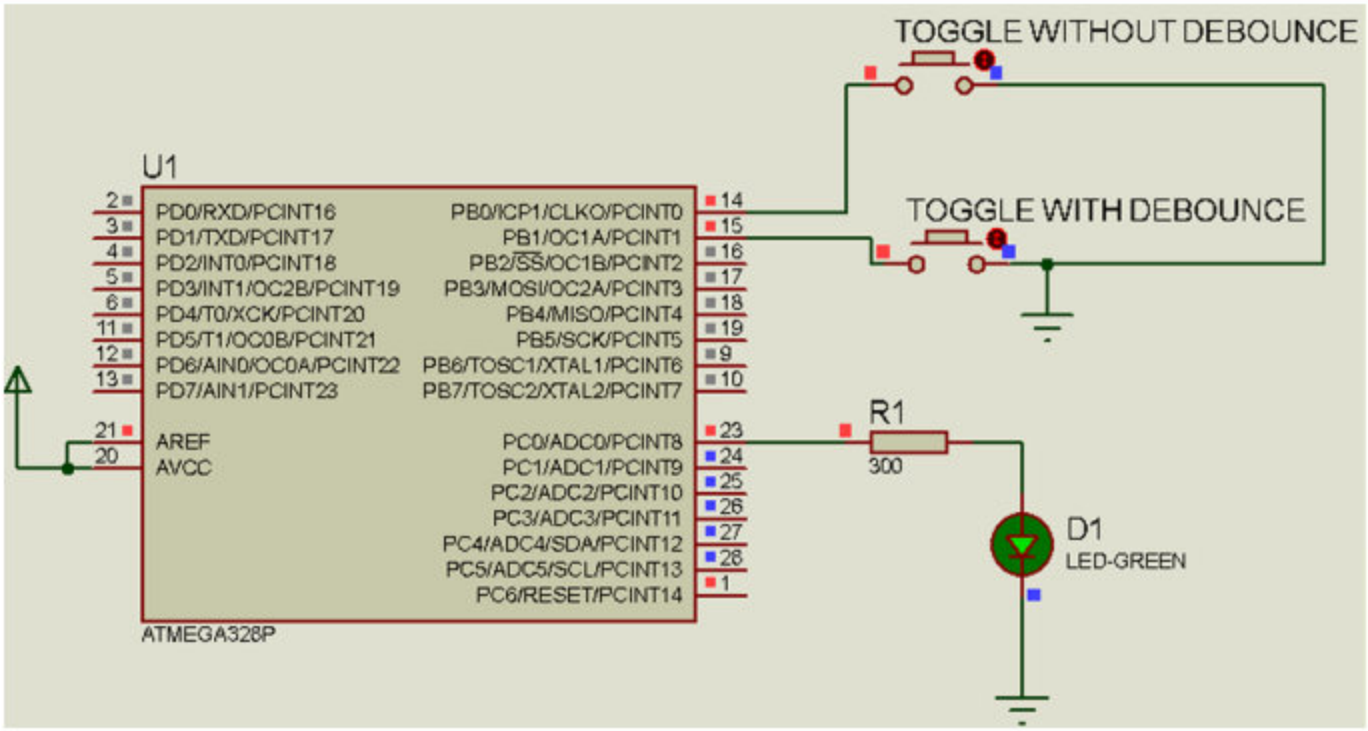
الصيغتان صحيحتان حيث يمكن كتابة رقم البت المراد اللإزاحة إليه ويمكن أيضاً كتابة اسم البت مثل PORTB2 أو DDD7 أو ..



كيف نعرف اسم البت ؟ موجود بصور المسجلات من Datasheet المتحكم (الأشكال 5 / 6 / 7)

التطبيق الرابع :

معالجة أثر الاهتزاز في المفاتيح الميكانيكية Debounce



الكود:

```
#include <avr/io.h>
#define F_CPU 1000000
#include <util/delay.h>

int main(void)
{
   DDRB=0x00; //port B input
   DDRC=0xFF; //port C output

   PORTB |= (1 << PORTB0)|(1<<PORTB1); //activation of pull up resistance on PB0 and PB1

    while (1)
    {
        /*    Toggle Without debounce    */
      if ((PINB & (1<<0))==0)
         {PORTC ^= (1<<0);} //Toggle PC0

        /*     Toggle With debounce     */
      if ((PINB & (1<<PINB1))==0)
         {_delay_ms(25);}
      if ((PINB & (1<<PINB1)) == 0)
         { PORTC ^= (1<<0);} //Toggle PC0
          else { }
    }
}
```

شرح الكود السابق :



1. قبل الـ main : تضمين المكتبات و تحديد تردد المهتز الداخلي 1MHz

2. داخل main :

- تعيين منافذ البوابة B كمداخل .

- تعيين منافذ البوابة C كمخارج .

- تفعيل PULL UP على المدخلين PB0 و PB1

3. داخل While :

- الجزء الأول اختبار قيمة المنفذ PB0 ، عند الضغط على الزر يصبح صفر فيتم عمل toggle (قلب) لحالة PC0 أيLED ، وهنا لم تتم معالجة أثر الاهتزاز إذ أنه عند الضغط على الزر قد يحدث بعض الاهتزاز بأجزاء من الثانية فيفهم المتحكم أنه تم الضغط أكثر من مرة أو أنه حدثت ضغطة عابرة ، بالتالي تنقلب حالة الـLED أكثر من مرة سريعاً

- في الجزء الثاني تم حل هذه المشكلة باختبار الزر الموصول إلى PB1 فيما إذا كان مضغوطاً أم لا وعند الضغط عليه ينتظر لمدة قصيرة (25 ms) ثم يختبر الزر مرة أخرى فإذا كان لا يزال مضغوطاً فهذا يعني أنها ضغطة طبيعية وليست عابرة أو من أثر الاهتزاز ، فيقلب حالة الـLED (toggle) بدون مشاكل .



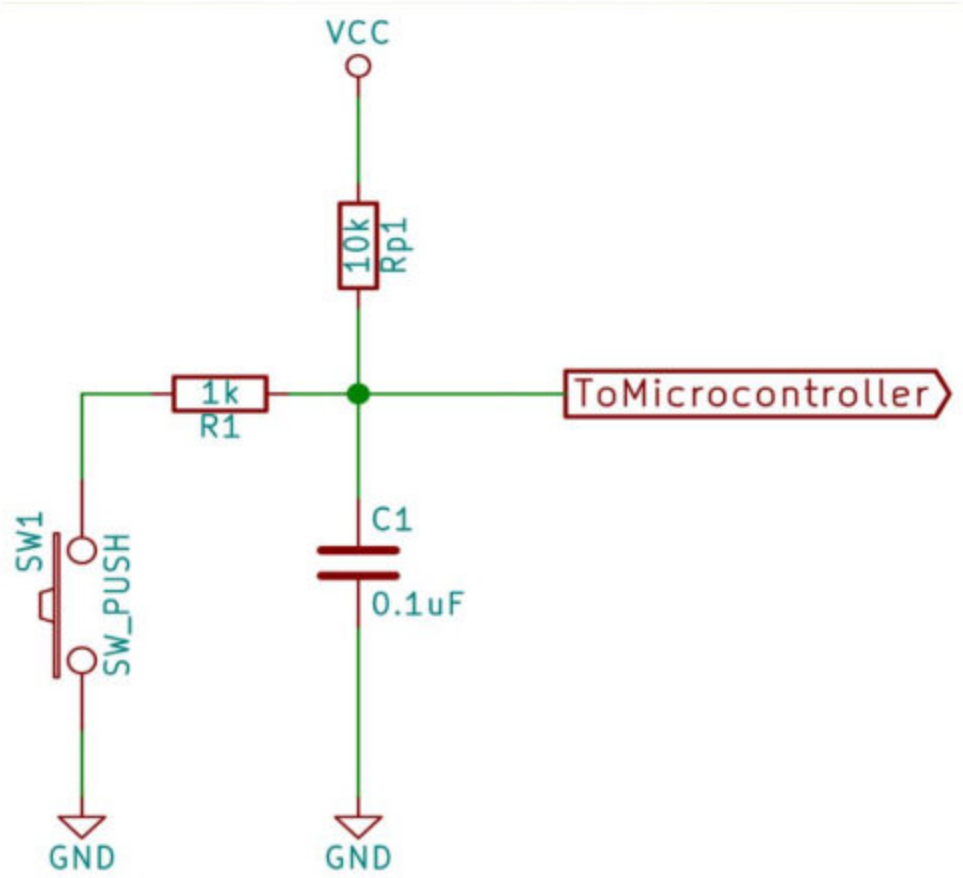
وقف وقف شفتك !!

قلتلي "يلا ح قوم نام هلأ وبفيق ع الفجر بكمل دراسة " ما!

مسيلمة الكذاب قوم اعمل فنجان قهوة وكمل يلا ورانا تخرج

معالجة أثر الاهتزاز الميكانيكي إلكترونياً

يمكن معالجة أثر الاهتزاز بدون تعديل الكود وذلك عن طريق دارة الكترونية بسيطة تضاف بين الزر و المتحكم:

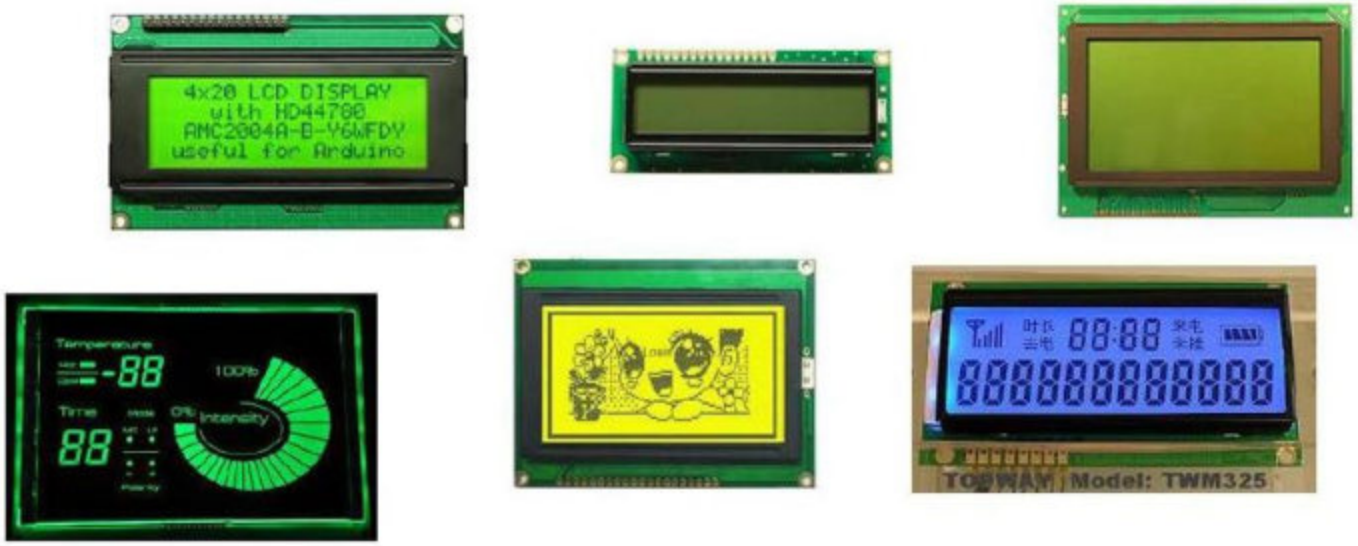


ملاحظة : { اهتزاز ≡ bounce ، معالجة أثر الاهتزاز ≡ debounce }

ملاحظة : تظهر مشكلة الاهتزاز في التطبيق السابق فقط في زر toggle بينما في زر set و reset لا يهم إن حدث اهتزاز أم لا ، لأنه في زر set إذا جاءت نبضة للمتحكم سيعمل set للLED ، حتى إذا جاءته أكثر من نبضة أو نبضات عابرة لن يتغير الأمر و سيقوم بعمل set بجميع الأحوال ، الأمر ذاته بالنسبة لزر reset ولكن مدخل الـ toggle إذا جاءه عدد زوجي من النبضات (بسبب النبضات العابرة) فلن يقوم بوظيفته ولن يعكس حالة الLED .

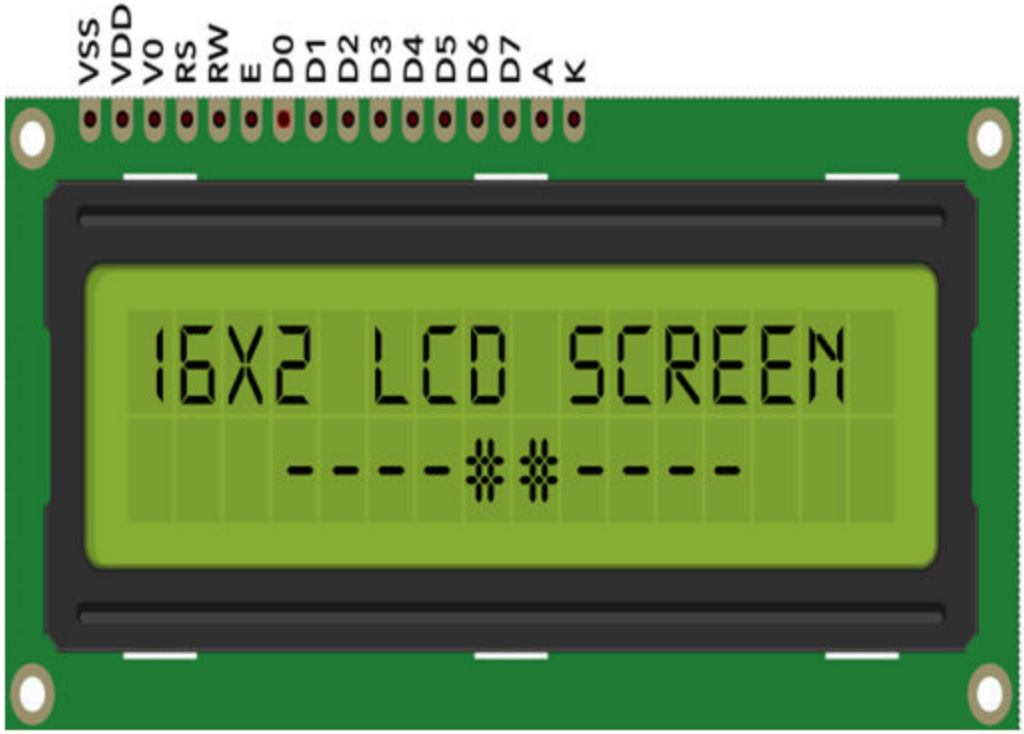
التطبيق الخامس :

التعامل مع شاشات العرض البلورية LCD :



أشكال مختلفة لشاشات العرض البللورية LCD

مخطط توزيع الأرجل للشاشة 2 × 16



No	Symbol	Function
1	VSS	Ground
2	VDD	5V +
3	V0	Contrast
4	RS	Register
5	RW	Read/Write
6	E	Enable
7	D0	Data bus
8	D1	Data bus
9	D2	Data bus
10	D3	Data bus
11	D4	Data bus
12	D5	Data bus
13	D6	Data bus
14	D7	Data bus
15	A	Anode (5V+)
16	K	Cathode (GND)

جدول الرموز : Font Table

هذا الجدول يوضح القيم الثنائية Binary للأرجل من D0 إلى D7 التي تمثل كل حرف أو رمز داخل ذاكرة الشاشة ، حيث أن الشاشة لا تستقبل الحرف نفسه مثل "A" بل تستقبل كود يقابله حرف معين ...

مثلاً عند إرسال القيمة 01000001 (أو 41 بالسداسي عشر) يظهر الحرف A

عند إرسال القيمة 01000010 أو (42 بالسداسي عشر) يظهر الحرف B



طريقة قراءة الجدول :

-الصفوف (على اليسار) تمثل أول 4 بتات $b3 \leftarrow b0$

-الأعمدة (في الأعلى) تمثل آخر 4 بتات $b7 \leftarrow b4$

فيكون الحرف A هو تقاطع الصف 0100 مع العمود 0001 أي :

0B 01000001 = 0X41



الشكل 10



الشكل 11

جدول الأوامر Command Table :

هذا الجدول يحتوي على الأوامر التي ترسلها إلى سجل التعليمات للتحكم بالشاشة .

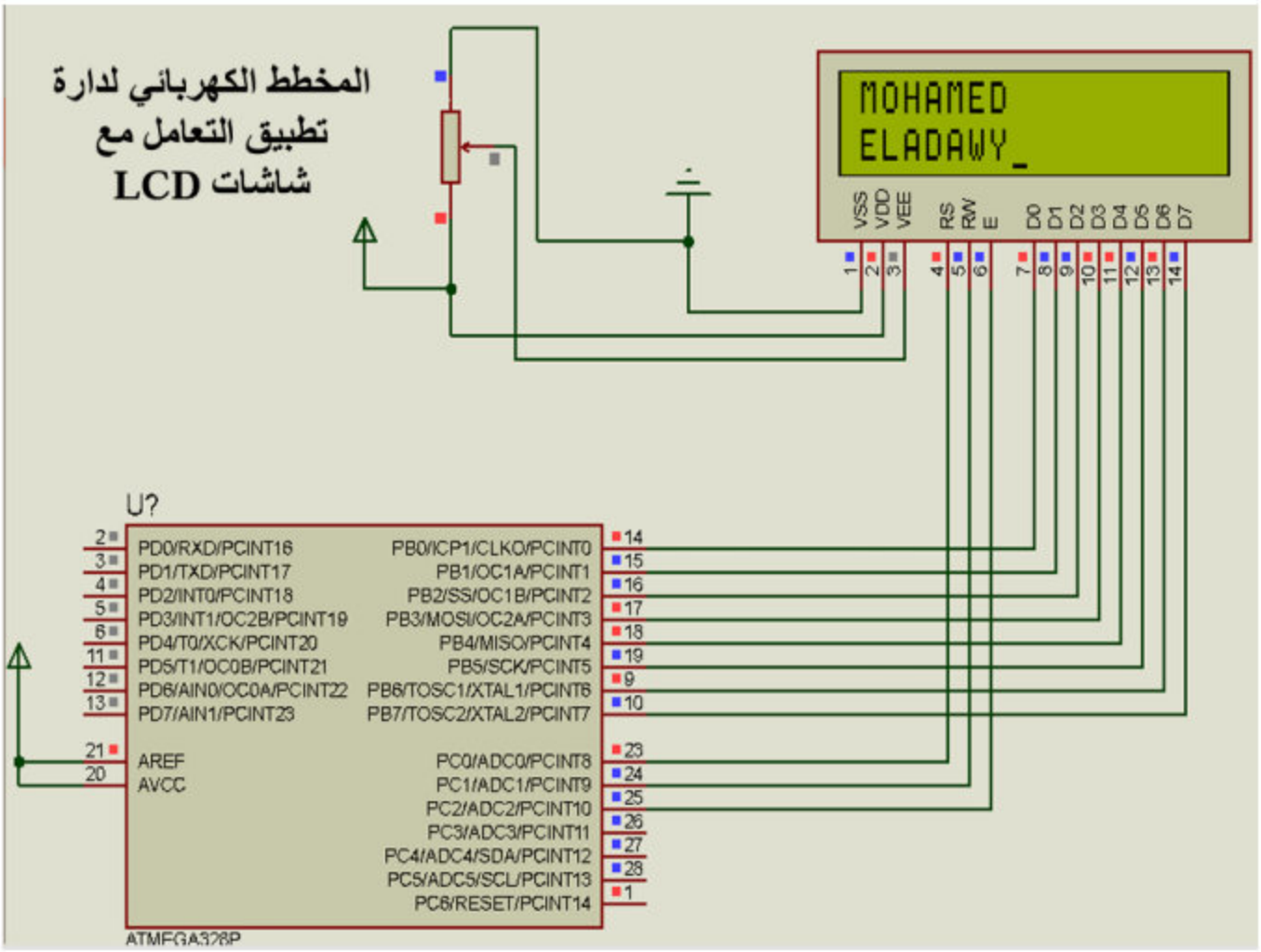
Code (Hex)	Command to LCD Instruction Register
1	Clear Display screen
2	Return home
4	Decrement cursor (Shift cursor to left)
6	Increment cursor (Shift cursor to Right)
5	Shift display right
7	Shift display left
8	Display off, cursor off
A	Display on, cursor off
C	Display on, cursor off
E	Display on, cursor blinking
F	Display on, cursor blinking
10	Shift cursor position to left
14	Shift cursor position to right
18	Shift the entire display to the left
1C	Shift the entire display to the right
80	Force cursor to beginning of 1st line
C0	Force cursor to beginning of 2nd line
38	2 lines and 5x7 Matrix www.embetronicx.com

ملاحظة :

أجداول ليست للحفظ !

في الامتحان ، يمكن فرضها بشكل مدروس في حال لم تعطى من قبل الدكتور بورقة مستقلة ..

المخطط الكهربائي لدارة تطبيق التعامل مع شاشات LCD :



الكود الخاص بالمثال العملي على تطبيقات شاشة *LCD* :

```
#include <avr/io.h>                 // تضمين مكتبة التحكم بمنافذ AVR
#define F_CPU 1000000               // تعريف تردد المعالج 1MHZ
#include <util/delay.h>             // تضمين مكتبة التأخير

int main(void)                      // بداية البرنامج الرئيسي
{
    DDRB=0xFF;                      // جعل جميع منافذ PORTB خارج و هي المنافذ الخاصة بالبيانات
    DDRC=0xFF;                      // جعل جميع منافذ PORTC خارج و هي المنافذ الخاصة بالتحكم

    //**** Display Initialization تهيئة الشاشة***//

    PORTC=0x00;                     // RS=0 وضع الأوامر، R/W=0 وضع الكتابة، Enable=0
    PORTB= 0x38;                    /* ترسل الكود 38 على خطوط البيانات الذي يعني أن الشاشة مؤلفة من
                                       5x7 سطرين و حجم مصفوفة النقاط التي يرسم بها كل حرف هو*/
    PORTC |=(1<<PORTC2);            // جعل Enable = 1 لارسال الامر
    PORTC &=~(1<<PORTC2);           // جعل Enable = 0 لتثبيت الامر
    _delay_ms(1000);                // انتظار 1 ثانية

    PORTB= 0x01;                    // ترسل الكود 01 على خطوط البيانات لمسح محتويات الشاشة
    PORTC |=(1<<PORTC2);            // جعل Enable = 1 لارسال الامر
    PORTC &=~(1<<PORTC2);           // جعل Enable = 0 لتثبيت الامر
    _delay_ms(1000);                // انتظار 1 ثانية

    PORTB= 0x0f;                    // تشغيل الشاشة مع إظهار المؤشر بوميض
    PORTC |=(1<<PORTC2);            // جعل Enable = 1 لارسال الامر
    PORTC &=~(1<<PORTC2);           // جعل Enable = 0 لتثبيت الامر
    _delay_ms(1000);                // انتظار 1 ثانية

    PORTB=0x80;                     // نقل المؤشر إلى بداية السطر الأول
    PORTC |=(1<<PORTC2);            // جعل Enable = 1 لارسال الامر
    PORTC &=~(1<<PORTC2);           // جعل Enable = 0 لتثبيت الامر
    _delay_ms(500);                 // انتظار نصف ثانية
```

```
//****** First Name ( Mohamed ) Mohamedطباعة كلمة  *****//

PORTC |=(1<<PORTC0);              // RS=1 للدخول إلى وضع إرسال البيانات المراد طباعتها

PORTB=0x4d;                       // ترسل الكود المكافئ للحرف Mعلى خطوط البيانات
PORTC |=(1<<PORTC2);              // جعل Enable = 1 لارسال الامر
PORTC &=~(1<<PORTC2);             // جعل Enable = 0 لتثبيت الامر
_delay_ms(1000);                  // انتظار

PORTB=0x4f;                       // ترسل الكود المكافئ للحرف Oعلى خطوط البيانات
PORTC |=(1<<PORTC2);              // جعل Enable = 1 لارسال الامر
PORTC &=~(1<<PORTC2);
_delay_ms(1000);

PORTB=0x48;                       // الحرف H
PORTC |=(1<<PORTC2);              // جعل Enable = 1 لارسال الامر
PORTC &=~(1<<PORTC2);             // جعل Enable = 0 لتثبيت الامر
_delay_ms(1000);

PORTB=0x41;                       // الحرف A
PORTC |=(1<<PORTC2);              // جعل Enable = 1 لارسال الامر
PORTC &=~(1<<PORTC2);             // جعل Enable = 0 لتثبيت الامر
_delay_ms(1000);

PORTB=0x4d;                       // الحرف M
PORTC |=(1<<PORTC2);              // جعل Enable = 1 لارسال الامر
PORTC &=~(1<<PORTC2);             // جعل Enable = 0 لتثبيت الامر
_delay_ms(1000);

PORTB=0x45;                       // الحرف E
PORTC |=(1<<PORTC2);              // جعل Enable = 1 لارسال الامر
PORTC &=~(1<<PORTC2);             // جعل Enable = 0 لتثبيت الامر
_delay_ms(1000);


PORTB=0x44;                       // الحرف D
PORTC |=(1<<PORTC2);              // جعل Enable = 1 لارسال الامر
PORTC &=~(1<<PORTC2);             // جعل Enable = 0 لتثبيت الامر
_delay_ms(1000);

//**** New Line***//

PORTC=0x00;                       // RS=0 للعودة إلى وضع الأوامر
PORTB=0xC0;                       // نقل المؤشر إلى بداية السطر الثاني
PORTC |=(1<<PORTC2);              // جعل Enable = 1 لارسال الامر
PORTC &=~(1<<PORTC2);             // جعل Enable = 0 لتثبيت الامر
_delay_ms(500);                   // انتظار نصف ثانية

//**** Last Name ( Eladawy ) ***//

PORTC |=(1<<PORTC0);              // RS=1 وضع إرسال بيانات

PORTB=0x45;                       // الحرف E
PORTC |=(1<<PORTC2);              // جعل Enable = 1 لارسال الامر
PORTC &=~(1<<PORTC2);             // جعل Enable = 0 لتثبيت الامر
_delay_ms(1000);

/*   الامر نفسه ينطبق على الاحرف الاخرى   */

PORTB=0x59;                       // الحرف Y
PORTC |=(1<<PORTC2);
PORTC &=~(1<<PORTC2);
_delay_ms(1000);

while (1)
{
}
}
```



إنشاء مكتبة للتعامل مع الشاشات :

لتسجيل العمل البرمجي يتم إنشاء مكتبات خاصة و تضمينها ضمن بيئة برمجة المتحكمات واستخدامها عند الحاجة .

في المثال التالي تم إنشاء مكتبة باسم LCDlib وهذه الأوامر الأساسية الخاصة بها:

تهيئة الشاشة	LCD_Init();
مسح محتويات الشاشة	LCD_Clear();
الانتقال إلى السطر الأول	LCD_GoToLineOne();
الانتقال إلى السطر الثاني	LCD_GoToLineTow();
وضع المؤشر بمكان محدد	LCD_GoToXY(row,col);
لطباعة حرف واحد فقط	LCD_DataWrite(data);
لطباعة string	LCD_DisplayString(data);

المثال :

```
#define F_CPU 1000000ul
#include <avr/io.h>
#include <util/delay.h>
#include "LCDlib.h"                    //تضمين مكتبة LCD الجاهزة

int main(void)
{
    LCD_Init();                        //تهيئة شاشة LCD

    LCD_DataWrite('H');                // عرض الحرف H
    LCD_DataWrite('e');                // عرض الحرف e
    LCD_DataWrite('l');                // عرض الحرف l
    LCD_DataWrite('l');                // عرض الحرف l
    LCD_DataWrite('o');                // عرض الحرف o

    LCD_GoToLineTwo();                 // الانتقال إلى السطر الثاني

    LCD_DisplayString("World");        // عرض الكلمة World

    while (1)
    {
    }
}
```

يقوم الكود السابق بطباعة عبارة" Hello World "على الشاشة حيث يطبع كلمة Hello حرفاً حرفاً و كلمة World كاملة من أبا التنوّع في التوابع المستخدمة في المكتبة .

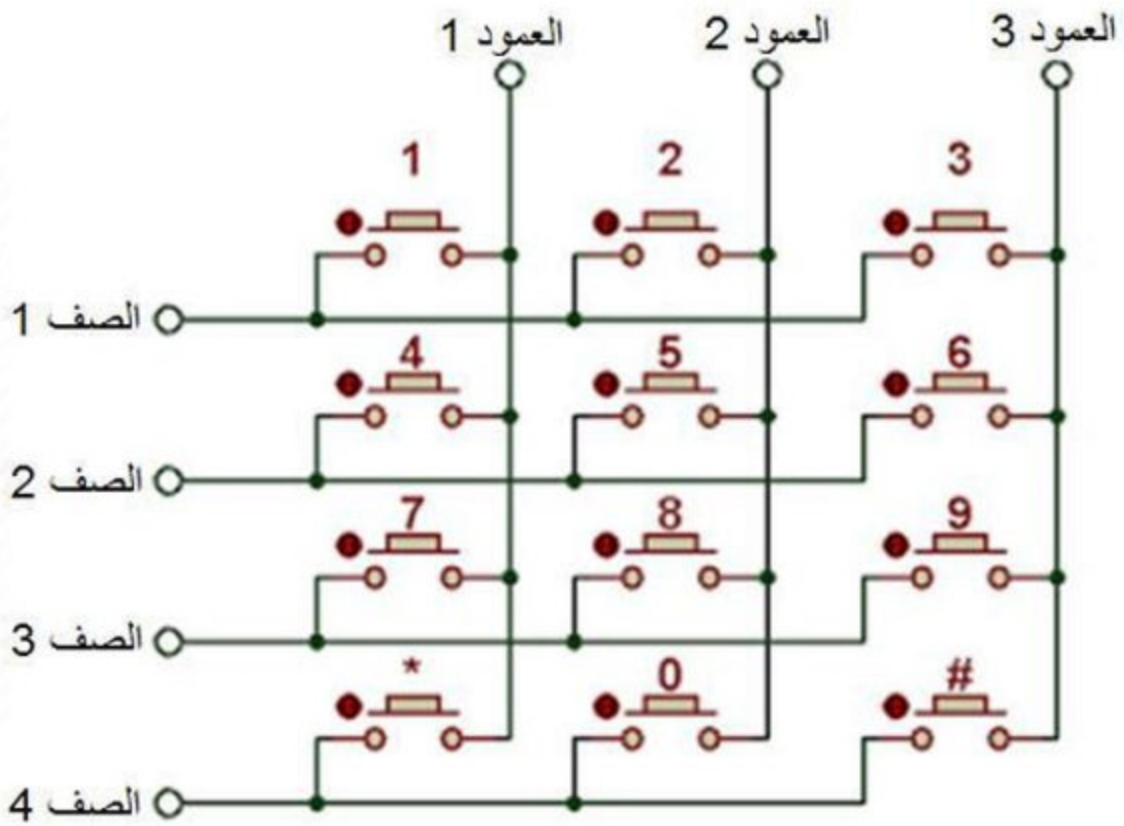


ونلاحظ الفرق الكبير في الكلفة البرمجية مقارنة مع المثال السابق .

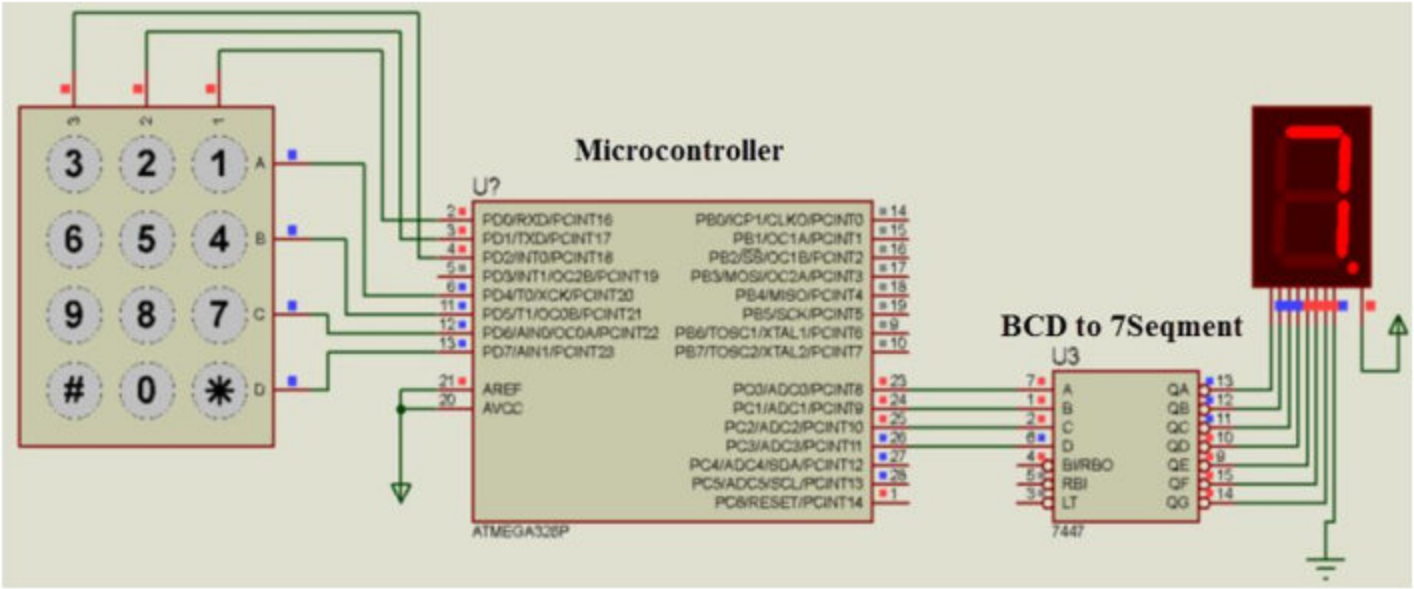
التطبيق السادس :

التعامل مع لوحة المفاتيح Keypad :





مخطط توصيل التطبيق :



كود التطبيق :

```
#include <avr/io.h>
#define F_CPU 1000000UL
#include <util/delay.h>

#define ROW PORTD               // PORTD الصفوف موصولة على
#define COL PIND                // PIND قراءة الأعمدة من

unsigned char key;              // لتخزين قراءة الأعمدة
unsigned char ScanKey;          // يستخدم لتفعيل الصفوف واحدًا تلو الآخر للفحص
unsigned char i;                // عداد الحلقة

int main(void)
{
    DDRD = 0xF0;                // PD4-PD7 مخارج للصفوف و PD0-PD2 مداخل للاعمدة
    DDRC = 0xFF;                // جعل منافذ C مخارج من اجل الشاشة

    while (1)
    {
        // انتظار حتى ضغط مفتاح
        do
        {
            ROW = 0x0F;                 // جعل جميع الصفوف LOW
            key = COL & 0x07;           // قراءة الأعمدة فقط
        }
        while (key != 0x07);            // لا يخرج حتى يتم الضغط

        // Debounce إزالة الاهتزاز
        do
        {
            do
            {
                ROW = 0x07;             // تثبيت الصفوف
                key = COL & 0x07;       // قراءة الأعمدة
            }
            while (key == 0x07);        // الانتظار حتى التأكد من الضغط

            _delay_ms(1);               // تأخير إزالة الاهتزاز

            ROW = 0x07;
            key = COL & 0x07;

        }
        while (key == 0x07);

        // بدء البحث عن الصف المضغوط
        ScanKey = 0xE0;

        for(i = 0; i < 0X04; i++)
        {
            ROW = ScanKey + 0x07;       // تفعيل صف واحد فقط
            key = COL & 0x07;           // قراءة الأعمدة

            if(key != 0x07)             // إذا تم اكتشاف ضغط
            break;

            ScanKey = (ScanKey << 1) + 0x10; // الانتقال للصف التالي
        }

        key = key + (ScanKey & 0xF7); // تكوين الشيفرة النهائية

        switch(key)
        {
            case 0xE6: PORTC = 0x01; break; // 1 زر
            case 0xE5: PORTC = 0x02; break; // 2 زر
            case 0xE3: PORTC = 0x03; break; // 3 زر

            case 0xD6: PORTC = 0x04; break; // 4 زر
            case 0xD5: PORTC = 0x05; break; // 5 زر
            case 0xD3: PORTC = 0x06; break; // 6 زر
```



```
            case 0xB6: PORTC = 0x07; break; // 7 زر
            case 0xB5: PORTC = 0x08; break; // 8 زر
            case 0xB3: PORTC = 0x09; break; // 9 زر

            case 0x76: PORTC = 0x0A; break; // * زر
            case 0x75: PORTC = 0x00; break; // 0 زر
            case 0x73: PORTC = 0x0B; break; // # زر

            default: PORTC = 0x00;
        }
    }
}
```

شرح الكود :

بالنسبة لجزء التخلص من الاهتزازات التي تنشأ عند الضغط على أي مفتاح ميكانيكي فإن الحلقة الأولى do..while تضع أصفاراً على كل الصفوف وواحدات على الأعمدة بالرقم 0x07 ، ثم قراءة الصفوف بالأمر key = COL & 0x07; وطالما أن قيمة المفتاح تكون هي القيمة 0x07 فإن ذلك يعني أنه ليس هناك مفتاح مضغوط فلا يعمل شيء و يظل يدور في هذه الحلقة. إذا تم ضغط أي مفتاح فإن قيمة المفتاح ستختلف عن القيمة 0x07، وبالتالي يخرج البرنامج من هذه الحلقة .

بعد الخروج من الحلقة يتم عمل تأخير مقداره واحد ميلي ثانية (1ms) ، ثم وضع الرقم 0x07 مرة ثانية على الصفوف و الأعمدة وقراءة الأعمدة مرة ثانية بنفس الأمر key = COL & 0x07; ، فإذا رجعت قراءة المفتاح Key إلى القيمة 0x07 مرة ثانية ، فإن ذلك يدل على أن الذي حدث كان مجرد هزة مفتاح، وبناءً عليه يعود البرنامج للدوران في الحلقة do...while الأولى ، وأما إذا ظلت قيمة المفتاح key مختلفة عن القيمة 0x07 فإن ذلك يعني أنه تم بالفعل ضغط أحد المفاتيح ، وعليه فإن البرنامج سيخرج من الحلقة وينتقل إلى مرحلة تحديد المفتاح المضغوط ثم مرحلة فك شيفرته .



عندما يصل البرنامج لهذه المرحلة فإن ذلك يعني أن هناك مفتاح قد تم ضغطه ، ولكننا لا نعرف أي واحد من المفاتيح الاثني عشرة هو . نفترض أننا قمنا بضغط المفتاح 5 الموجود عند تقاطع الصف الثاني و العمود الثاني . في البداية يتم وضع المتغير ScanKey = 0xe0 ، حيث 0xe0=11100000 مما يعني وضع الصف الأول بالقيمة صفر (باللون الأحمر) . ثم الدخول في حلقة for() من أربع مرات . في الأمر الأول في هذه الحلقة يتم تنفيذ الأمر التالي :

ROW=ScanKey + 0x07; حيث ستكون القيمة ROW التي سيتم إخراجها على البوابة D هي :

11100000 + 00001111 = 11100111 = 0xe0 + 0x07 . بعد ذلك يتم تنفيذ الأمر التالي مباشرة key = COL & 0x07; وحيث أن الصف الأول فقط هو الفعال (يساوي الصفر) ، فإن قراءة الأعمدة في هذه الحالة ستكون COL=00000111، أي 0x07 ، وبالتالي فإن نتيجة عملية AND ستكون هي نفسها القيمة 0x07 ، وستكون قيمة المفتاح key = 0x07 ، وبالتالي ستكون

نتيجة تنفيذ الأمر الشرطي if غير محققة ، و سينتقل التنفيذ إلى الأمر التالي وهو :

ScanKey=(ScanKey<<1)+0x10

هذا الأمر سيزيح القيمة ScanKey ناحية اليسار بمقدار بت واحدة ، وبالتالي فإن نتيجة تنفيذ هذا الأمر ستصبح كالتالي : 11010000= 00010000 + 11000000 ، أي أن الصف الثاني أصبح الآن يساوي الصفر (الأحمر) . بعد ذلك يرجع التنفيذ لبداية الحلقة for حيث ينفذ الأمر ROW=ScanKey + 0x07; ، الذي سيعطي ROW=11010000+00000111=11010111 . بعد ذلك يتم تنفيذ الأمر key = COL & 0x07 حيث القيمة COL=00000101 نتيجة ضغط الزر 5 في العمود الثاني و في الصف الثاني الذي يساوي صفر من الأمر السابق .

وعلى ذلك ستكون قيمة المفتاح Key=00000101 أي لا تساوي 0x07 ، وبهذا ستكون نتيجة تنفيذ أمر الشرط if() التالي حقيقية ، وبالتالي يتم تنفيذ الأمر break; الذي ينتج عنه الخروج من الحلقة for() وينفذ أول أمر بعد الحلقة وهو key = key + (ScanKey & 0xf7); الذي ينتج عنه أن قيمة المفتاح ستصبح :

Key=00000101 + (11010000&11110111)=11010101=0xd5 وهذه هي شيفرة المفتاح التي تحتوي الصف والعمود الواقع فيه هذا المفتاح . بعد ذلك يدخل البرنامج في فك شيفرة المفتاح للتعرف عليه من خلال الأمر switch(key){ } حيث سيذهب إلى الحالة d5 التي ستجعل البوابة PORTC=05 التي ستظهر على المظهر ذو السبع قطع كما في البرنامج . بنفس الطريقة يمكن تتبع أي ضغطة لأي مفتاح .



حاول كتابة البرنامج و تنفيذه والتحقق من صحته .

المقاطعات

تعريف المقاطعة

هي طريقة تجعل المتحكم يوقف ما يفعله مؤقتاً عندما يحدث حدث مهم ، ثم ينفذ كوداً خاصاً لمعالجة هذا الحدث المهم وبعدها يعود ليكمل عمله الطبيعي .

أنواع المقاطعات :



✓ مقاطعات خارجية :

- مقاطعة التصفير Reset
- مقاطعة الطلب الخارجي INT0 – INT7

✓ مقاطعات داخلية :

- مقاطعة المؤقتات / العدادات
- مقاطعة حادثة المسك للمؤقتات / العدادات
- مقاطعة اكتمال التحويل للADC
- مقاطعة اكتمال الارسال للنافذة التسلسلية SPI
- مقاطعة النافذة التسلسلية USART
- مقاطعة المقارن التشابهي
- مقاطعة النافذة التسلسلية TWI

المقاطعات الخارجية في متحكمات AVR

سؤال 1 : كيف يعرف المتحكم مكان برنامج خدمة المقاطعة ؟

:)

- يحتوي المتحكم داخله على مكان يسمى متجه المقاطعة interrupt vector وهذا المكان يحوي عنوان بداية برنامج ISR لكل مقاطعة يمكن أن يتعامل معها هذا المتحكم .
- عادة يشغل متجه مقاطعة أول 1KB من ذاكرة البرمجة للمتحكم .
- عند حدوث أي مقاطعة يقفز المتحكم فوراً إلى مكان مخصص لهذه المقاطعة في متجه المقاطعة، ليعرف منه عنوان أول أمر في برنامج خدمة المقاطعة ISR فيذهب إليه ويبدأ بتنفيذه .
- ينتهي برنامج خدمة المقاطعة بالأمر RET1 الذي يجعل المتحكم يعود إلى نفس المكان الذي خرج منه في البرنامج الأساسي .

سؤال 2 : كيف يعود المتحكم إلى نفس المكان الذي خرج منه في البرنامج الأساسي بعد تنفيذ برنامج خدمة المقاطعة ؟

:)

- يوجد في المتحكم سجل هام يسمى PC(Program Counter) يحفظ عنوان التعليمة التالية التي سيقوم المعالج بتنفيذها.
- عند حدوث مقاطعة يضع المتحكم قيمة PC داخل المكدسة Stack ، وهي عبارة عن جزء من الذاكرة العشوائية RAM .
- يذهب المتحكم إلى برنامج خدمة المقاطعة ISR وينفذ أوامره .
- في نهاية برنامج خدمة المقاطعة ينفذ المتحكم تعليمة RET1 فيقوم المتحكم بسحب عنوان العودة من Stack ووضعه مرة أخرى في PC فيكمل البرنامج من نفس المكان الذي توقف عنده تماماً .

الخطوات التي ينفذها المتحكم عند حدوث أي مقاطعة :

1. ينتهي المتحكم من تنفيذ الأمر الحالي الذي يقوم بتنفيذه .
2. بحسب نوع المقاطعة يقفز البرنامج إلى مكان محدد في متجه المقاطعة ليعرف منه عنوان برنامج خدمة المقاطعة ISR
3. لكي يقبل المتحكم أي مقاطعة يجب أن يكون علم هذه المقاطعة I (Interrupt flag) فعّالاً (يساوي الواحد) و يتم ذلك قبل طلب المقاطعة .
4. أيضاً كل مقاطعة لها علم تنشيط خاص بها flag وهذا العلم لا بد من تنشيطه حتى تقبل هذه المقاطعة
5. بناءً على حالة العلمين السابقين يتم قبول المقاطعة ، عندها يقوم المعالج بدفع عداد البرنامج PC إلى المكدسة .
6. يقفز المتحكم إلى برنامج خدمة المقاطعة ISR وبمجرد دخوله فيه يقوم بتصفير علم المقاطعة I ليمنع أي مقاطعة أخرى ، لكن يمكن للمستخدم داخل برنامج خدمة المقاطعة أن يعيد تنشيط علم المقاطعة I ليسمح للمقاطعات الأخرى - إن حدثت- أن تخرج بعملية التنفيذ من برنامج الخدمة الحالي لخدمة مقاطعات أخرى .
7. في نهاية برنامج خدمة المقاطعة ينفذ المتحكم الأمر RET1 فيقوم المتحكم بسحب عنوان العودة من stack ووضعه داخل PC مرة أخرى ليعود المتحكم إلى البرنامج الأساسي
8. يقوم المتحكم بتنشيط علم المقاطعة مرة أخرى حتى يكون جاهزاً لاستقبال المقاطعات من جديد .

ماذا سيحدث لو حدثت مقاطعتان للمتحكم في نفس الوقت بالصدفة ؟

إذا حدث ذلك فإن المتحكم يخدم المقاطعة ذات الأولوية الأعلى

- **لدينا في جدول متجه المقاطعة للمتحكم Atmega328 المقاطعات مرتبة من الأكثر أولوية حتى الأقل أولوية :**

متجه المقاطعة interrupt vector للمتحكم atmega328

رقم المتجه	العنوان فى ذاكرة البرمجة	مصدر المقاطعة	الوصف
1	0x0000	RESET	يمكن لهذه المقاطعة أن تحدث من الطرف reset، أو بعد تشغيل القدرة، أو بسبب دائرة مراقبة القدرة brown out، أو بسبب مؤقت كلب الحراسة
2	0x0002	INT0	طلب مقاطعة من على الطرف INT0 للشريحة
3	0x0004	INT1	طلب مقاطعة من على الطرف INT1 للشريحة
4	0x0006	PCINT0	المقاطعة رقم 0 بسبب تغير فى جهد الطرف
5	0x0008	PCINT1	المقاطعة رقم 1 بسبب تغير فى جهد الطرف
6	0x000A	PCINT2	المقاطعة رقم 2 بسبب تغير فى جهد الطرف
7	0x000C	WDT	المقاطعة بسبب فيضان فى مؤقت كلب الحراسة
8	0x000E	TIMER2_COMPA	تطابق مقارنة A فى المؤقت ٢
9	0x0010	TIMER2_COMPB	تطابق مقارنة B فى المؤقت ٢
10	0x0012	TIMER2_OVF	فيضان فى المؤقت ٢
11	0x0014	TIMER1_CAPT	التقاط حدث للمؤقت١
12	0x0016	TIMER1_COMPA	تطابق مقارنة A فى المؤقت ١
13	0x0018	TIMER1_COMPB	تطابق مقارنة B فى المؤقت ١
14	0x001A	TIMER1_OVF	فيضان فى المؤقت ١
15	0x001C	TIMER0_COMPA	تطابق مقارنة A فى المؤقت 0
16	0x001E	TIMER0_COMPB	تطابق مقارنة B فى المؤقت 0
17	0x0020	TIMER0_OVF	فيضان فى المؤقت 0
18	0x0022	SPI STC	اكتمال الإرسال التتابعى بالنظام SPI
19	0x0024	USART_RX	اكتمال الاستقبال التتابعى Rx فى النظام USART
20	0x0026	USART_UDRE	مسجل البيانات فاضى فى النظام USART
21	0x0028	USART_TX	اكتمال الإرسال التتابعى Tx فى النظام USART
22	0x002A	ADC	اكتمال التحويل فى المحول التماثلى الرقمى ADC
23	0x002C	EE READY	الذاكرة EEPROM جاهزة
24	0x002E	ANALOG COMP	المقارن التماثلى
25	0x0030	TWI	الواجهة مع نظام السلكين للتراسل I2C
26	0x0032	SPM READY	ذاكرة تخزين البرنامج جاهزة



ملاحظة :

هذا الجدول ليس للحفظ!، وإنما يجب فهم أن مقاطعة Reset هي الأكثر أولوية ثم يأتي بعدها المقاطعات الخارجية من ثم الداخلية .

المقاطعات الخارجية (INT0 و INT1) في المتحكم Atmega328

مسجل قناع المقاطعة الخارجية *EIMSK* :

المسؤول عن تفعيل أو تعطيل المقاطعات الخارجية INT0 و INT1

7	6	5	4	3	2	1	0
-	-	-	-	-	-	INT1	INT0

مثلاً لتفعيل INT0 نضع 1 في البت 0 :

```
EIMSK |= (1<<INT0);
```

ولتفعيل INT1 نضع 1 في البت 1 :

```
EIMSK |= (1<<INT1);
```

مسجل تحكم المقاطعة الخارجية EICRA :

المسؤول عن تحديد نمط عمل المقاطعة الخارجية بحيث :

1. تقدح عند الجبهة الصاعدة Rising edge
2. تقدح عند الجبهة الهابطة falling edge
3. تقدح عند المستوى المنخفض low level
4. تقدح عند تغير المستوى level change



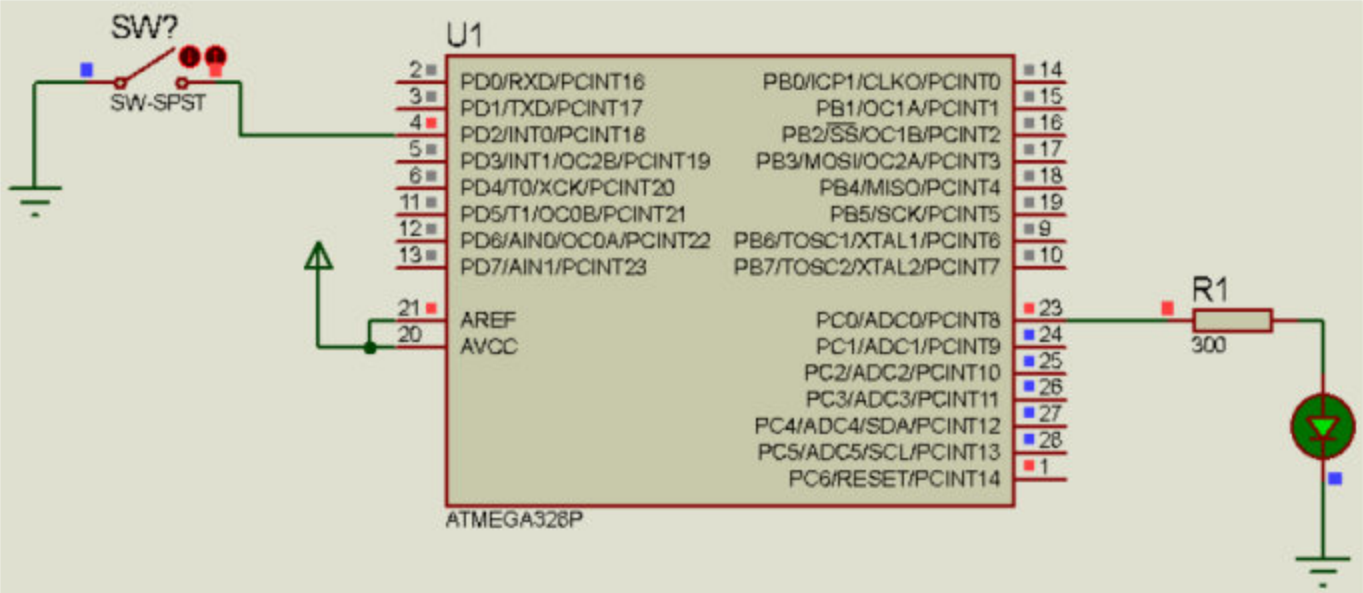
7	6	5	4	3	2	1	0
-	-	-	-	ISC11	ISC10	ISC01	ISC00

ويتم تحديد بتات هذا المسجل حسب نمط العمل المطلوب وفق الجدول الآتي :

ISC01 ISC11	ISC00 ISC10	الوصف (حالة الإشارة التى ستسبب المقاطعة)
0 0	0 0	يتم طلب المقاطعة على الخط INT0 بإشارة منخفضة المستوى (صفر وتظل صفر إلى أن تقبل) يتم طلب المقاطعة على الخط INT1 بإشارة منخفضة المستوى (صفر وتظل صفر إلى أن تقبل)
0 0	1 1	أى تغير منطقى على الخط INT0 (صفر إلى واحد، أو واحد إلى صفر) يتسبب فى المقاطعة أى تغير منطقى على الخط INT1 (صفر إلى واحد، أو واحد إلى صفر) يتسبب فى المقاطعة
1 1	0 0	الحافة الهابطة (واحد إلى صفر) على الخط INT0 تتسبب فى المقاطعة الحافة الهابطة (واحد إلى صفر) على الخط INT1 تتسبب فى المقاطعة
1 1	1 1	الحافة الصاعدة (صفر إلى واحد) على الخط INT0 تتسبب فى المقاطعة الحافة الصاعدة (صفر إلى واحد) على الخط INT1 تتسبب فى المقاطعة

مثال على المقاطعة :

الدارة:



الكود :

```
#include <avr/io.h>           // يحتوي تعريفات منافذ ومسجلات متحكم AVR
#include <avr/interrupt.h>    //  تضمين مكتبة المقاطعة

int main(void)
{
    DDRD &= ~(1<<DDD2);           // جعل الطرف PD2 دخل لأنه طرف المقاطعة INT0 نفسه

    /* طرف المقاطعة يجب أن يكون دخل حتى يقبل المقاطعة عليه */

    PORTD |= (1<<PORTD2);         // تفعيل المقاومة الداخلية Pull-up على الطرف PD2
    EICRA |= (1<<ISC00);          // تحديد نمط عمل المقاطعة INT0
    /* ISC00=1 و ISC01=0
       أي تعمل المقاطعة عند أي تغير منطقي سواء من 0→1 أو من 1→0
    */
    EIMSK |= (1<<INT0);           // تفعيل المقاطعة الخارجية INT0

    sei();                        /* Global Interrupt Enable تفعيل المقاطعات العامة
                                     يسمح للمعالج باستقبال جميع المقاطعات المفعلة*/

    DDRC = 0xFF;                  // جعل جميع أطراف PORTC مخارج
    PORTC = 0x01;                 // جعل PORTC0 = 1 أي تشغيل الدايود الموصول على PC0

    while (1)
    {
        // البرنامج الرئيسي فارغ و المتحكم يبقى بانتظار حدوث مقاطعة فقط
    }

}

ISR (INT0_vect)               // بداية برنامج خدمة المقاطعة الخاصة بـ INT0
{
    PORTC ^= (1<<0);              // عكس حالة البت PC0
    // إذا كان الدايود مضاء ينطفئ
    // وإذا كان مطفأ يضيء
}
```

البرنامج السابق يدور في الحلقة المغلقة while 1 ولا يفعل شيء ، وعند حدوث مقاطعة على الخط INT0 فإن المتحكم يقفز إلى برنامج خدمة المقاطعة (INT0_vect)

برنامج خدمة المقاطعة سيعكس حالة بت الخرج PC0 الموصول إلى LED وذلك عند حدوث أي تغير منطقي على خط المقاطعة INT0 .





تم بعون الله الانتهاء من بحث متحكمات الـAVR ...

اكتب ملاحظاتك:

فريقنا يتمنى لكم دراسة موفقة ❤